

Verification Continuum™

Verdi® Debug Decision Tree User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	7
Synopsys Statement on Inclusivity and Diversity	8

1. Overview	9
--------------------------	----------

2. Debug Decision Tree	10
Debug Decision Tree Usage	10
Environment Setup	10
Launch DDT	11
Add a Node by Selecting a Library	12
Set up the Global Variables	12
General Variables	13
Environment Variables	13
Regular Expressions	14
Signal Mappings	14
Data Flow and Packet	17
Fill in Arguments of a Node	17
Mode Decide	18
Fill in Node's Properties	19
Fit Window, Zoom in, Zoom out, Select Nodes, Move Nodes	20
Duplicate a Node(s)	21
Connect Two Nodes	21
Use Selection Pane to Select Nodes	23
GUI Preference	23
Save or Restore a DDT Session	24
Session Changed Hint	25
Run a DDT Session	26
Batch Run	26
Run via Command	26
Run on GUI	28
Regression Run	28
Run via Command	28
Run on GUI	29
Organize Run Report	30

Contents

Debug Decision Tree Library	32
Specify Attributes and Signals	33
Textlog Attributes	33
Waveform Signals	34
Search Range	35
Line Range	35
Time Range	35
Value Compare	35
Textlog Value Compare	36
Waveform Value Compare	36
Enum	37
UVM Severity	37
Text-Log Libraries	37
Enum	37
Library	38
Waveform Libraries	43
Enum	43
Library	44
Signature Libraries	47
Weight	47
Encode Option Usage	47
Enum	47
Library	48
Miscellaneous Libraries	53
Library	53
Customize Library	55
Generate Library Template	55
Make Customize Library	57
Example of Customize Library	59
Import a Customized Library	59
DDT_WORK_ROOT	60
Batch Mode	60
Available Commands	61
add_customized_lib(self , filename)	61
add_global_var_field(self , tab_type)	61
add_tree_node(self , module_path , type_name , node_name=None , pos=None)	61
auto_layout(self)	62
batch_run(self , summary_file=None , report_file=None , target=None)	62
close(self)	62

Contents

<code>collapse_all_nodes(self)</code>	62
<code>collapse_node(self, uid)</code>	62
<code>connect_tree_node(self, src_uid, tgt_uid)</code>	63
<code>decode_session(self, output_file, source_file=None, detail=False)</code>	63
<code>delete_ddt_nodes(self, uid_list)</code>	63
<code>delete_global_var_field(self, tab_type, field_idx)</code>	63
<code>delete_regrun_record(self, idx_list)</code>	64
<code>disconnect_tree_node(self, src_uid, tgt_uid)</code>	64
<code>duplicate_ddt_nodes(self, uid_list)</code>	64
<code>exec_command(self, cmd)</code>	64
<code>expand_all_nodes(self)</code>	64
<code>expand_node(self, uid)</code>	64
<code>export_regrun_record(self, filename)</code>	65
<code>gen_lib_template_to_file(self, filename)</code>	65
<code>get_all_nodes(self)</code>	65
<code>get_global_var_count_in_tab(self, tab_type)</code>	65
<code>get_global_var_tab_field(self, tab_type, index)</code>	65
<code>get_node_child(self, uid)</code>	66
<code>get_node_config_by_arg(self, uid, arg_name)</code>	66
<code>get_node_description(self, uid)</code>	66
<code>get_node_name(self, uid)</code>	66
<code>get_node_parent(self, uid)</code>	66
<code>get_node_pos(self, uid)</code>	67
<code>get_node_property(self, uid, name)</code>	67
<code>get_node_size(self, uid)</code>	67
<code>get_regrun_record(self, row_idx=None)</code>	67
<code>import_regrun_record(self, filename)</code>	67
<code>load_preference(self, conf_file)</code>	68
<code>move_ddt_node(self, uid, pos)</code>	68
<code>redo(self)</code>	68
<code>register_macro_api(self, name, module_path, func_name)</code>	68
<code>remove_macro_api(self, name)</code>	68
<code>regression_run(self, config_idx=None, num_jobs=1, summary_file=None, report_file=None, target=None, log_file=None, record_file=None, force=False, throw_exception=False, **kwargs)</code>	69
<code>replay(self, cmd_file)</code>	69
<code>restore_global_var_tab(self, tab_type, filename)</code>	70
<code>restore_session(self, filename)</code>	70
<code>run_cli(self)</code>	70
<code>run_gui(self)</code>	70
<code>save_global_var_tab(self, tab_type, filename)</code>	70
<code>save_session(self, filename)</code>	71

Contents

set_configuration(self , settings)	71
set_global_var_tab_field(self , tab_type , index , var_name , value , add_quote=False)	71
set_node_color(self , uid , red , green , blue)	71
set_node_config_by_arg(self , uid , arg_name , value , add_quote=False)	72
set_node_description(self , uid , value)	72
set_node_name(self , uid , value)	72
set_node_property(self , uid , name , value , push_undo=True)	73
set_node_size(self , uid , width=0.0 , height=0.0)	73
undo(self)	73
update_regrun_record_by_idx(self , row_idx , tab_type , idx , data , add_quote=False)	74
update_regrun_record_by_name(self , row_idx , var_name , data , add_quote=False)	74
update_regrun_setting(self , setting)	74
Command File	74
Demo Examples	76
Customized Library	76
Copy Folder	76
Run Session	76
View Tree	76
Text-Log Parse	77
Copy Folder	77
Run Session	77
View Tree	77
Waveform Extraction	77
Copy Folder	77
Run Session	77
View Tree	78

Preface

Verdi HW/SW Debug User Guide describes the Verdi Hardware-Centric flow (HW)/ Software-Centric flow (SW) Debug Solution.

This chapter describes the following section:

- [Customer Support](#)
- [Synopsys Statement on Inclusivity and Diversity](#)

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customerfacing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Overview

The Verdi® Automated Debug Platform provides a unique approach to analog mixed-signal (AMS) verification that enables engineers to debug complex AMS designs. AMS debug features provide visibility into design hierarchy and source code for both analog and digital designs. When digital and analog FSDB files are loaded, you can view and debug analog signals through the annotated source code, schematic and waveform. This document focuses on the VHDL/Verilog-SPICE Mixed-Signal design scope.

2

Debug Decision Tree

Debug Decision Tree (DDT) has the following modules to perform debug operations in the DDT libraries:

- [Debug Decision Tree Usage](#)
- [Run a DDT Session](#)
- [Debug Decision Tree Library](#)
- [Import a Customized Library](#)
- [Batch Mode](#)
- [Demo Examples](#)

Debug Decision Tree Usage

This chapter contains the following topics:

Environment Setup

To set up/enable DDT, use the given environmental variables:

```
%> setenv VERDI_HOME <VERDI_HOME_path>
%> setenv PATH
$VERDI_HOME/bin:$VERDI_HOME/platform/linux64/Python/bin:$PATH
%> setenv PYTHONPATH $VERDI_HOME/share/NPI/python
```

If using SLSE (Suse OS), add the following environment variable:

```
%> setenv LD_LIBRARY_PATH $VERDI_HOME/etc/lib/lib_dbus:$LD_LIBRARY_PATH
```

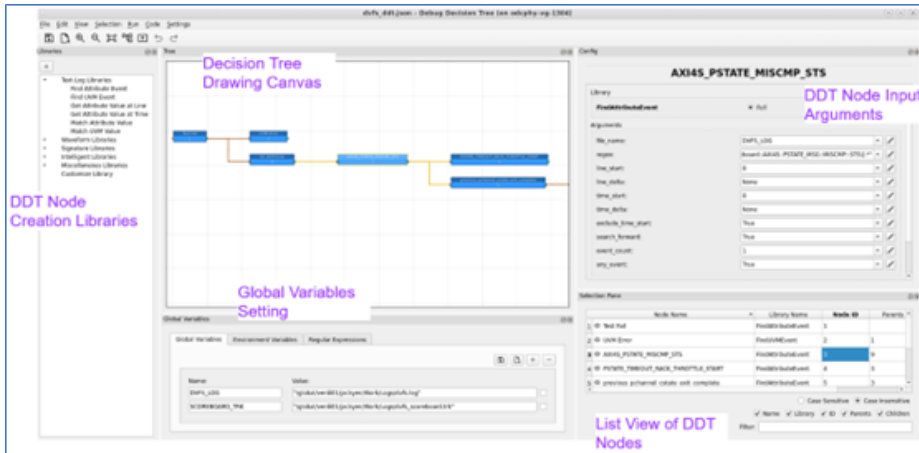
After making the setting (as mentioned above), the Python executable is set as following:

```
%> which python
<VERDI_HOME_path>/platform/linux64/Python/bin/python
```

Launch DDT

Run the command `ddt gui` to launch DDT GUI as following:

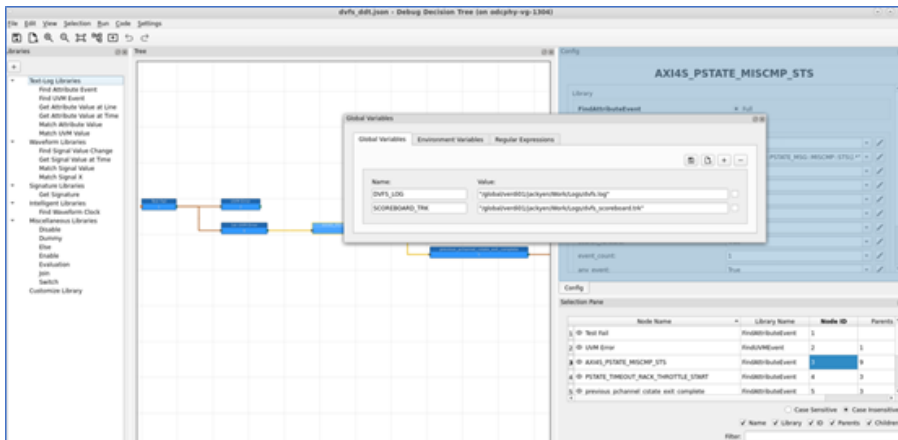
```
%> ddt gui &
```

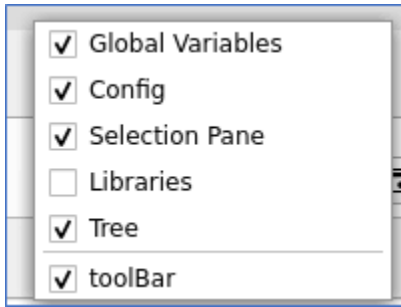


DDT has following five widgets:

1. Tree (decision tree drawing canvas)
2. Libraries
3. Global variables
4. Config (DDT node input arguments)
5. Selection Pane (list view of DDT nodes)

Use the mouse to drag any widget, and dock the widget to any place. You can also hide some of the widgets for your working mode.

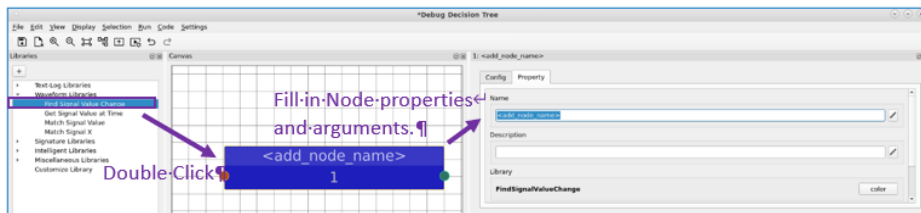




Add a Node by Selecting a Library

Use one of the predefined DDT libraries to create a DDT node through the following steps:

1. Expand the tree of the library category
2. Select a library, and double-click, a new DDT node will be added to the tree as shown in the following figure:



When you select the node, the corresponding **Config** tab will also be shown and the title of **Config** tab changes to **<Node ID>: <Node Name>**.

Under **Config** tab,

- Enter the node Name to a meaningful debug operation.
- Enter the input arguments for the node.

Note:

Additional information can be entered in **Description**, if the node name is not enough to describe the debug operation of the node.

Each library has its match function to determine the decision according to the input arguments. Details of the libraries will be described later.

Set up the Global Variables

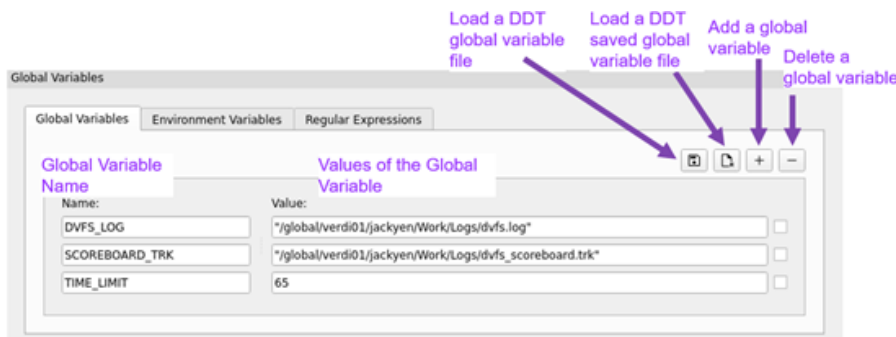
You can set up global variables for the entire DDT in the **Global Variables** tab. Usually, the global variables can be the log name, the FSDB name, the frequently used regular

expressions, the signal names in an FSDB, and known data that are expected to be used globally.

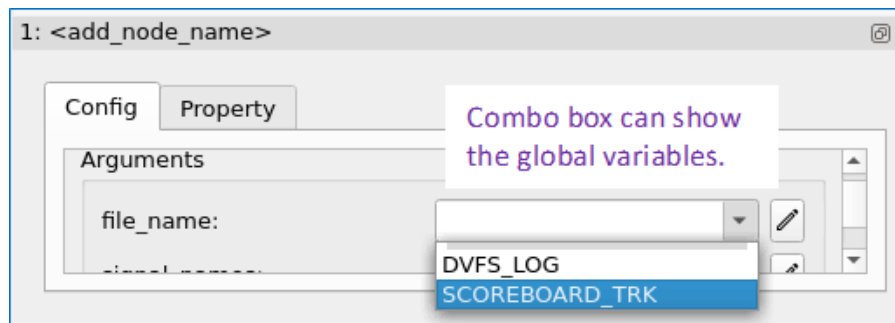
Use **Add** to add one more global variable and **Remove** to delete the selected global variable.

Note:

Under DDT, it must to use the quote (') or double quote (") to embrace string characters. DDT follows the Python syntax to set up the variables.



With the setting of global variables, when filling in the arguments of a node, clicking the combo box shows the available variables.



General Variables

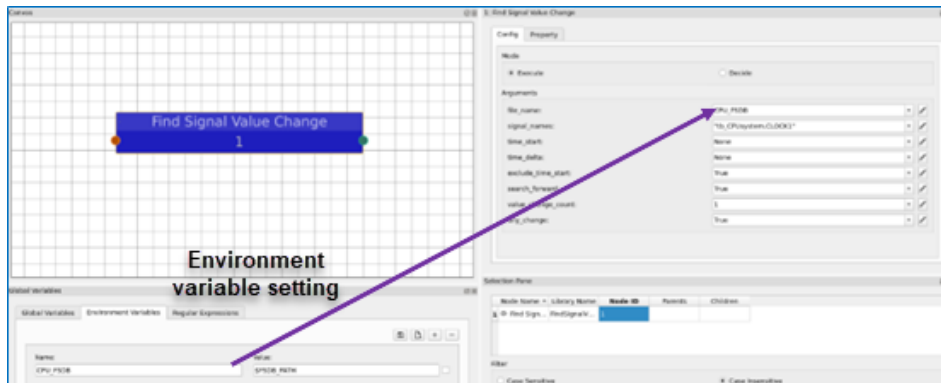
The left-hand-side name should be a valid Python identifier. The right-hand-side value should be a valid Python expression.

Note:

String variables should use single-quote ' or double-quote ".

Environment Variables

DDT allows to set up the variable to represent the environment variables. The usage is the same as [General Variables](#) where the right-hand-side value is evaluated by `os.path.expandvars()` during runtime.



Regular Expressions

DDT provides a separate tab to define variables of regular expressions. The usage is similar to [General Variables](#).

Signal Mappings

DDT provides a separate tab to define signal mappings, which helps users enter the field, `signal_names`, with ease.

The left-hand-side name should be a valid Python identifier, which is the name of the signal mapping. The right-hand-side value should be a `SignalMapping`. Every `SignalMapping` is a one-to-one mapping from a `str` to another `str` where keys are aliases and values are signal full name. An alias is usually a short name of a signal full name, enhancing readability.

After defining signal mappings, they can be selected by their name in the node config in signal-choosing fields in [Fill in Arguments of a Node](#). Instead of signal full names, signal aliases shows in the node result, which makes it easier to read. Furthermore, you can also access the values by the keys of `SignalMapping` as a dict.

For example, given a `SignalMapping`, `my_sig_map = {"r": "top.r", "e": "top.e"}`, the value, `"top.r"`, by `my_sig_map["r"]`.

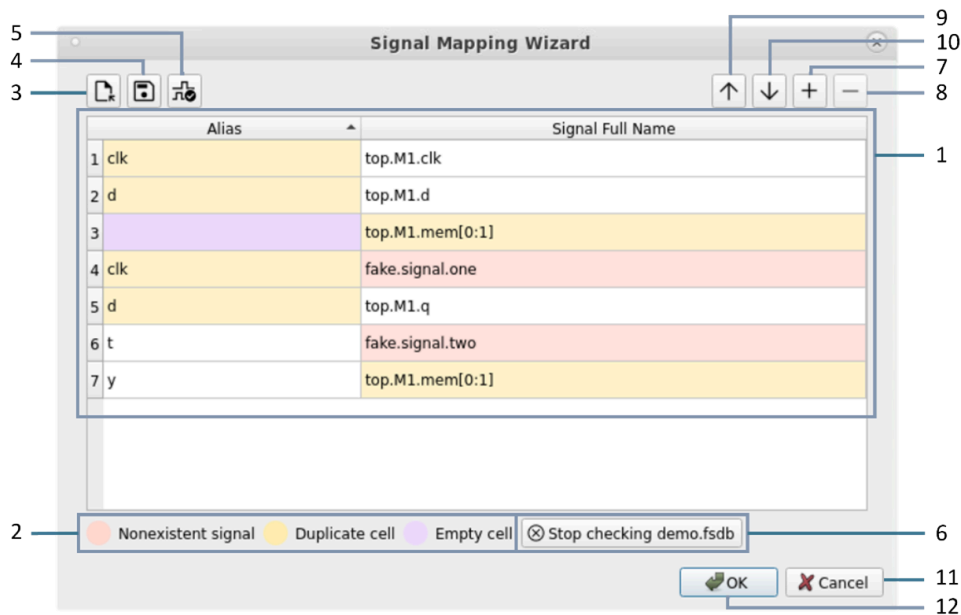
DDT provides **SignalMappingWizard** to help users to create valid signal mappings. To edit a signal mapping, click the  button to launch **SignalMappingWizard**.

SignalMappingWizard assists users edit signal mapping by providing an editor with a table view and validity checking. Users can manipulate the entries in the current mapping, such as moving, adding, deleting, sorting data. For validity checking, the wizard checks empty and duplicate entries by default. Users can also turn on the waveform sanity check function to check if signals exist in the provided waveform file.

Note:

SignalMappingWizard only accepts mapping that passes the validity check, in other words, there is no colored cell on the table.

To keep the status of **SignalMappingWizard**, users can also save the current status (📄) to a json file and load the saved json (📄) of the wizard. In addition, dragging signal(s) from **Verdi** and then dropping the signal full names to **SignalMappingWizard** is also supported.



SignalMappingWizard shown in the above image contains the following details:

- Shows current signal mapping content in table view.
- Legend shows error meaning of validity check.
 - signal does not exist in the provided waveform file.

Note:

This error shows when waveform sanity check (🔍) is turned on.

- duplicate signal name or alias.
- empty cell.

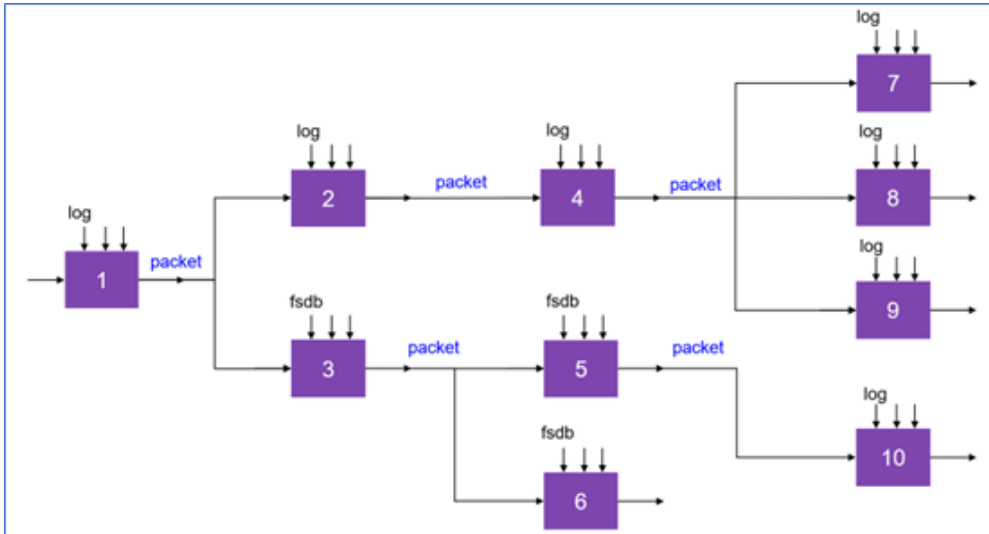
3.  Load signal mapping json file, the supported types of json files are as follows:
 - Signal mapping json file generated by **SignalMappingWizard**.
 - json file with a dictionary where keys are aliases and values are signal full names.
 - json file with a list of pair where the first element is alias, and the second element is signal full name.
4.  Save current signal mapping content to a json file.
5.  Turn on waveform sanity check function. The function checks if signals exist in the provided waveform file.
6. Turn off waveform sanity check function.
7. + Add a new row to the mapping.
8. - Remove selected row(s) from the mapping.
9. ↑ Move selected row(s) up.
10. ↓ Move selected row(s) down.
11. Close the wizard without saving the mapping.
12. Save the mapping.

Note:

The wizard only accept mapping that passes validity check.

Data Flow and Packet

The following picture shows the data transferred from parent to children node:



In DDT, we use a dictionary 'packet' to transmit the data. After evaluating the result of a parent node, the extracted data (e.g. the 'time' and the 'attribute', depends on which library used) will be put to the tree branch and its children's nodes can get the information. This is useful because in many situations the child node may evaluate some conditions followed by the data extracted from its parent.

When filling the arguments of a DDT node, for example, if a parent's packet contains the "time" field, the input of its child node can use below two ways to access the value of the "time" field:

- `packet["time"]`
- `B.time`

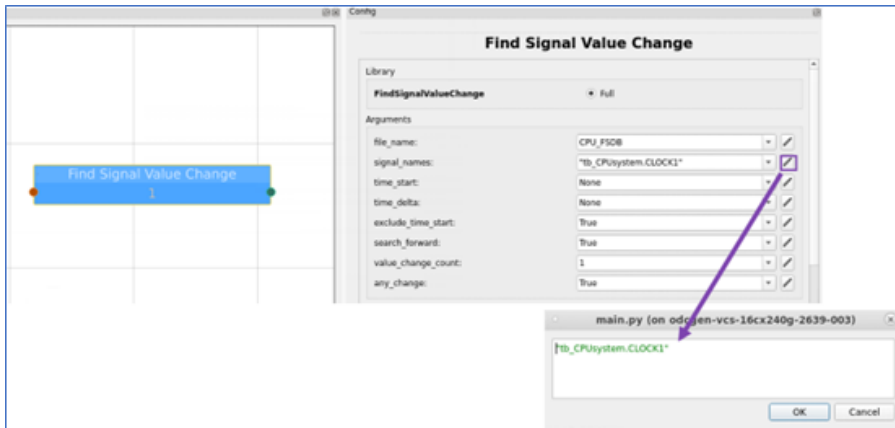
(The 'B' stands for 'branch' and can be changed in preference)

Fill in Arguments of a Node

Fill in the input arguments by editing texts or selecting the variables from the combo box. The available variables in the combo box comes from two sources as following:

1. The global variables (refer to [Set up the Global Variables](#)) which data type (string, int, and float etc.) matches the argument.
2. The parent's node output variables which data type matches the argument.

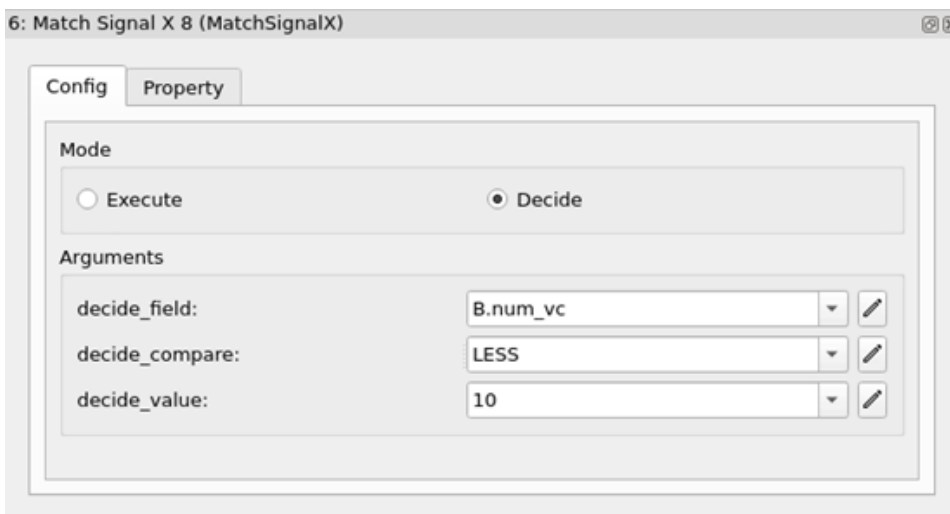
To edit the text, click **edit** icon. This icon will pop up a multiple-line editor. The editor is useful when you need to type a long text like regular expressions.



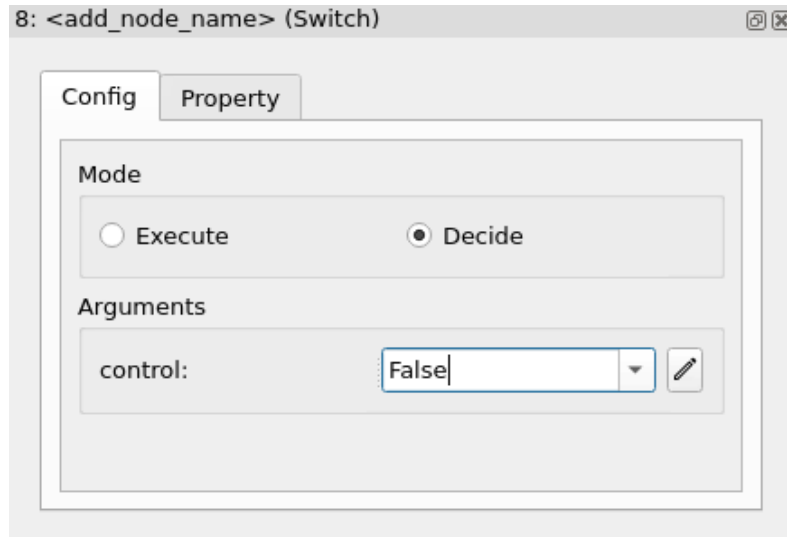
Mode Decide

In addition to filling argument values during execution, you can also change the default behavior whether DDT should run a node or not. By default, DDT runs a node if its parent node's execution result is `PASS`. For example, the `"result"` field from parent packet is `True`.

To change the default behavior of this decision, you can first click the `"Decide"` option in `"Mode"`, and then change the arguments accordingly. For example, changing a node's execution criteria to be checking a field `"num_vc"` from parent packet which value is less than a 10, you can fill in `"B.num_vc"`, `"LESS"`, and `"10"` in arguments `"decide_field"`, `"decide_compare"`, and `"decide_value"` accordingly.

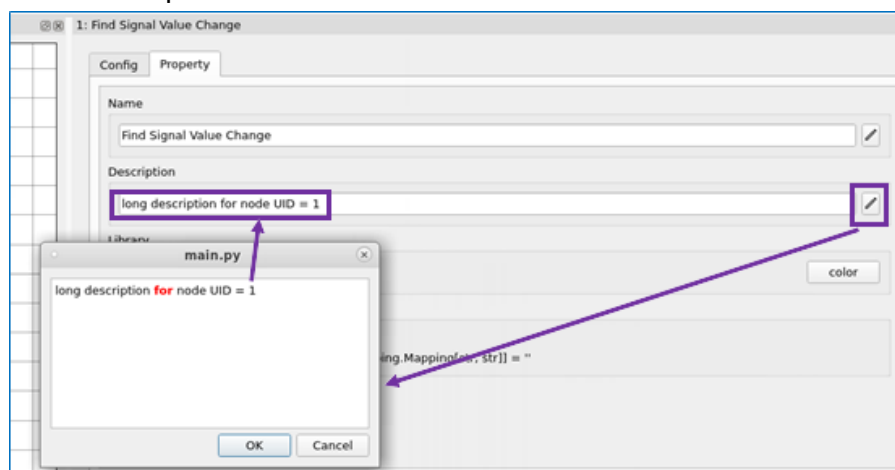


Different library may have different “Decide” arguments. For example, the “Switch” library in Miscellaneous category, its “Decide” argument has only one “control”, and it runs a node depending on the given “control” value.

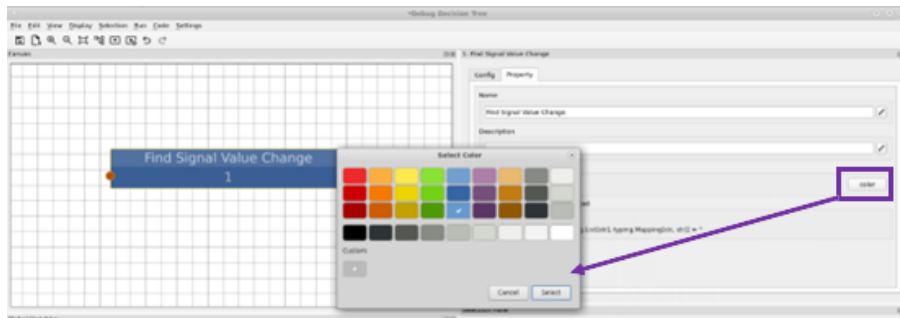


Fill in Node's Properties

For better readability and maintainability, add a description on each node. Thus, you do not need to worry about forgetting what the intension and behavior of the node. Similar to filling in the node arguments, you can add simple description to the line edit or long texts via the multiple line edit.

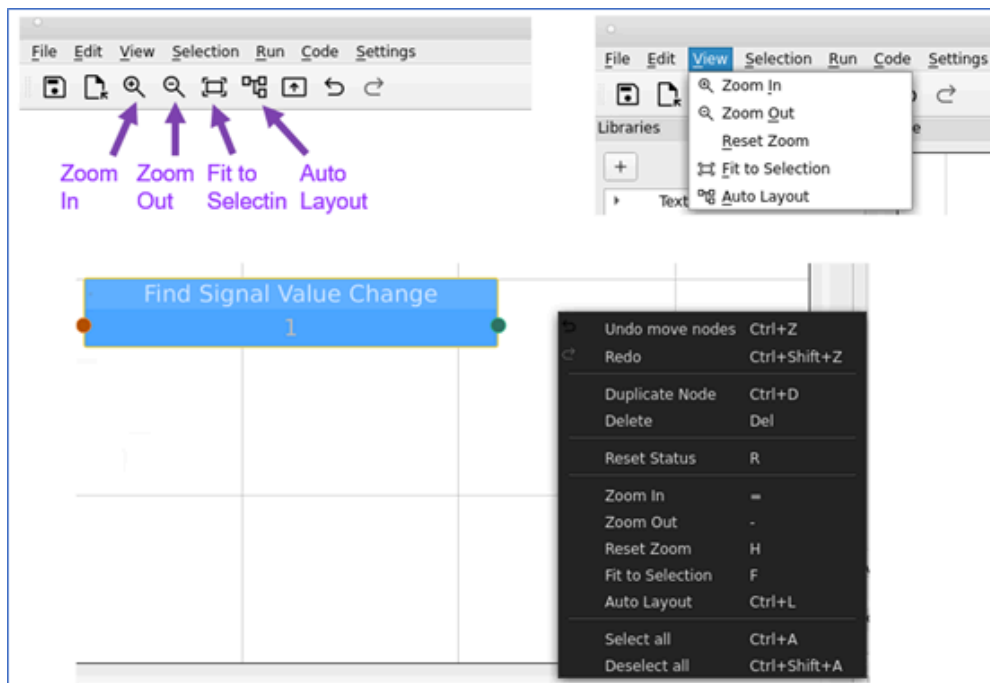


For some important nodes that you want to quickly locate from Canvas, change the node color from property tab. By clicking the *color* button, a color palette appears, and you can choose the color of your choice easily.



Fit Window, Zoom in, Zoom out, Select Nodes, Move Nodes

To view the DDT display functions, Use the menu 'View', click **Right Mouse Button** (RMB). Use the toolbar icon to **Zoom In / Zoom Out / Fit to Selection / Auto Layout**.



The hot keys are also shown in the menu of RMB. For example, you can use the hot key **f** to fit the DDT drawing window, or use **Ctrl+d** to duplicate a node.

Note:

The hot key character is **case insensitive**.

The operations by using the middle mouse are shown in the following table:

Note:

DDT uses the middle mouse button to move the drawing area.

DDT Operat ion	Middle Mouse Operation
Zoom In	Scroll Up the middle mouse wheel
Zoom Out	Scroll Down the middle mouse wheel
Move the DDT	Click the middle mouse button at any place of the drawing area and keep pressing the button; drag and finally release the button

Duplicate a Node(s)

Select one or many nodes, and use the RMB or the hotkey **Ctrl-D** to duplicate the node.

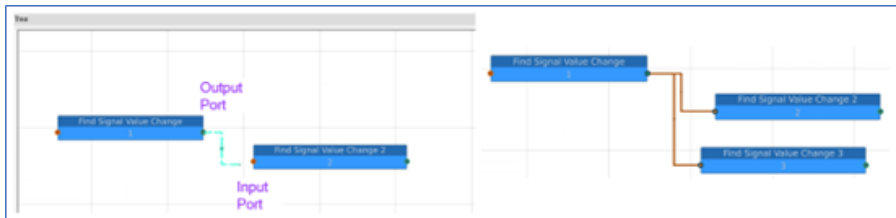
Connect Two Nodes

Each DDN contains following two ports:

1. Input port
2. Output port

Since the DDT logic is 'decision', the connection of two nodes must start from the Output port of one node to the Input port of the other node.

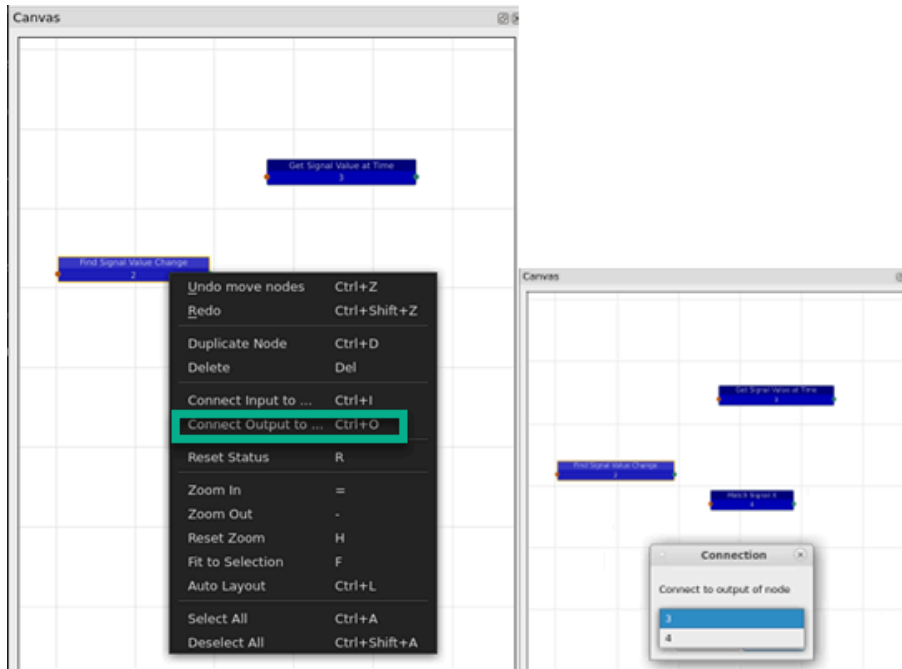
One Output port can connect to 'many' Input ports of other node (that is, one parent node connects to many children nodes). This means if the evaluation of a parent node is True, then the next step is to check all possible children nodes to see if they match the logic with specified input arguments.



You can also connect two nodes via the node's context menu by clicking the right mouse button on the node. Choosing **Connect Input to ...** or **Connect Output to ...** will pop

out a dialogue that allows the user to choose which node the user tends to create a connection from the drop-down menu.

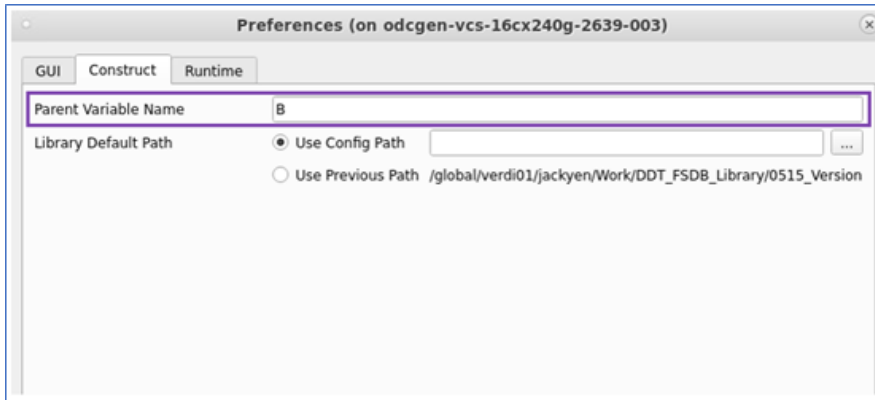
This pop-out dialogue is useful when the user finds it hard to find nodes within Canvas.



When connecting two nodes, in the Config Tab, the child node can see the output data passed by its parent. Those data will be displayed with the prefix **B.** in the combo box (**B** stands for **tree branch**). The following figure shows the **B.time** passed from a parent node to its children.

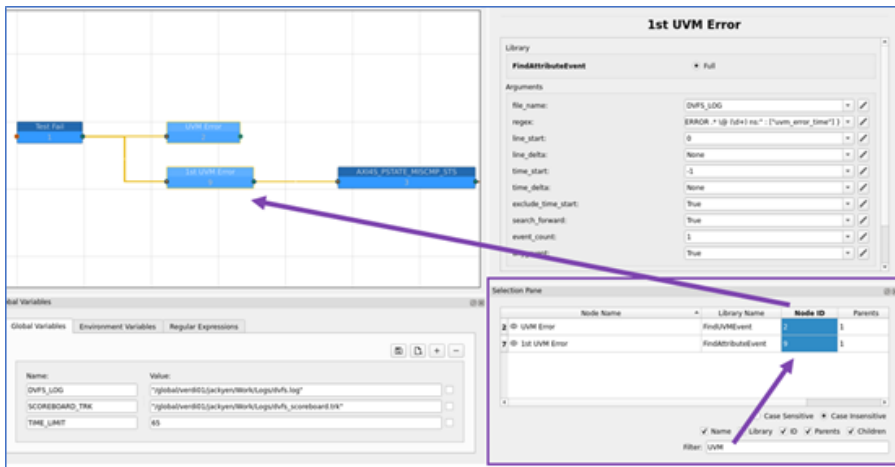


You can change the prefix from **B.** to any word in the menu **Preference**, and go to the **Construct** tab.



Use Selection Pane to Select Nodes

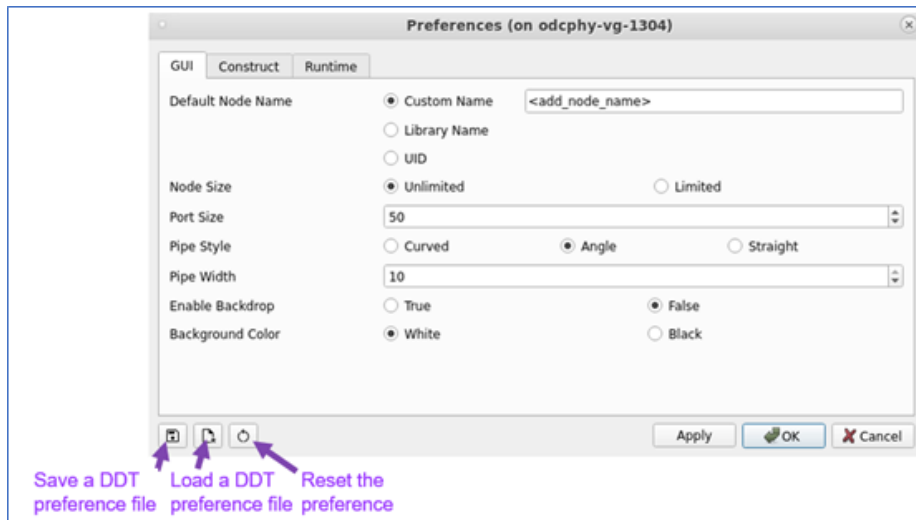
Use the **Selection Pane** to select one or many nodes. You can use the filter box to type the string that matches the name or ID of the nodes. The filter criteria in the **Selection Pane** use regular expression.



GUI Preference

Go to the menu **Preference** to set up the GUI preference, including the **default node name**, the **node size**, and the **background color** etc. You can save the preference to a file and load it back next time.

By default, DDT creates a preference file **ddt.conf** in your working directory. If you work in the same directory to open DDT GUI next time, the local preference file **ddt.conf** is loaded automatically.



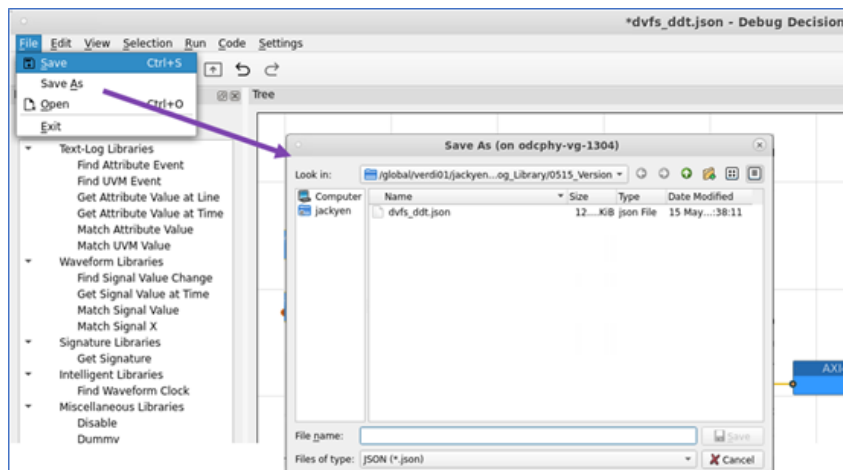
Use the command option **--preference <preference_file>** to specify the preference when launching DDT GUI. For example:

```
%> ddt gui --preference /tmp/my_ddt.conf &
```

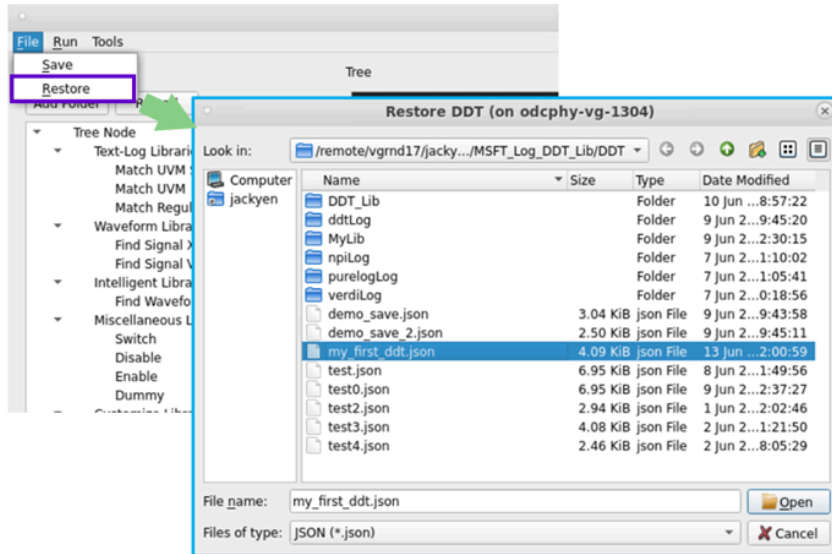
Save or Restore a DDT Session

To save and restore a DDT session, follow the given steps:

1. Click **File > Save/Save As** to save the DDT session (in JSON format).



2. Click **File > Restore** to restore the DDT.



Use the batch mode command `ddt gui <session>` to load the session. For example,

```
%> ddt gui dvfs_ddt.json &
```

Session Changed Hint

Once a session is edited, DDT will give a hint on window title. Based on the editing operations, it has two different hints: `*` and `?` like the below figures:

`?golden_session.json - Debug Decision Tree`

`*golden_session.json - Debug Decision Tree`

`?` means that all the edit operations only changing view-related information. Runtime behavior remains the same. For example, node location in canvas is changed, or some nodes is collapsed.

On the other hand, `*` represents some edit operations changes runtime behavior. For example, data in node config is changed or defined global variables are changed (add/delete/updated).

Run a DDT Session

After completing the creation of a DDT, you can run the created session for automatic diagnostic via DDT. Here DDT provides following two modes for running a session:

- Batch run
- Regression run

Batch run is responsible to run a session with settings provided in the session file and gives a run report for this run. You can identify the diagnostic failures from the run report. This also helps you to validate the settings of a constructed DDT. However, you must configure DDT several times if you have multiple errors waiting to be diagnosed. Thus, DDT provides another run mode named *regression run*.

Regression run mode allows users configuring multiple run settings at once, and then run all the settings sequentially or concurrently.

Batch Run

Batch run consists of the following sections:

Run via Command

Soon after the creation of a DDT, you can run the DDT by the command `ddt run <session>`. For example:

```
%> ddt run dvfs_ddt.json
```

The following snippet depicts running of a DDT session:

```
report log filename: report.log
Running node 1: Test Fail
Running node 2: UVM Error
Running node 3: AXI4S_PSTATE_MISCOMP_STS
WARNING: [VL APP] Time window is ignored because 'time' group is not specified in 'regex'.
Running node 4: PSTATE_TIMEOUT_RACK_THROTTLE_START
Running node 5: previous_pchannel_cstate_exit_complete
Running node 8: next_any_throttle_or_no_throttle_message_expected
Running node 9: 1st UVM Error
Running node 10: next_any_throttle_or_no_throttle_message_expected
summary log filename: summary.log
Test Fail: TRUE [1]
UVM Error: TRUE [2]
AXI4S_PSTATE_MISCOMP_STS: TRUE [3]
PSTATE_TIMEOUT_RACK_THROTTLE_START: FALSE [4]
previous_pchannel_cstate_exit_complete: TRUE [5]
next_any_throttle_or_no_throttle_message_expected: TRUE [8]
1st UVM Error: TRUE [9]
next_any_throttle_or_no_throttle_message_expected: TRUE [10]
```

The DDT run generates the following two output files:

1. *summary.log*: Details the execution summary of the DDT. The *summary.log* is in JSON format.
2. *report.log*: Is the dumped execution log (depends on the logging of the used libraries).

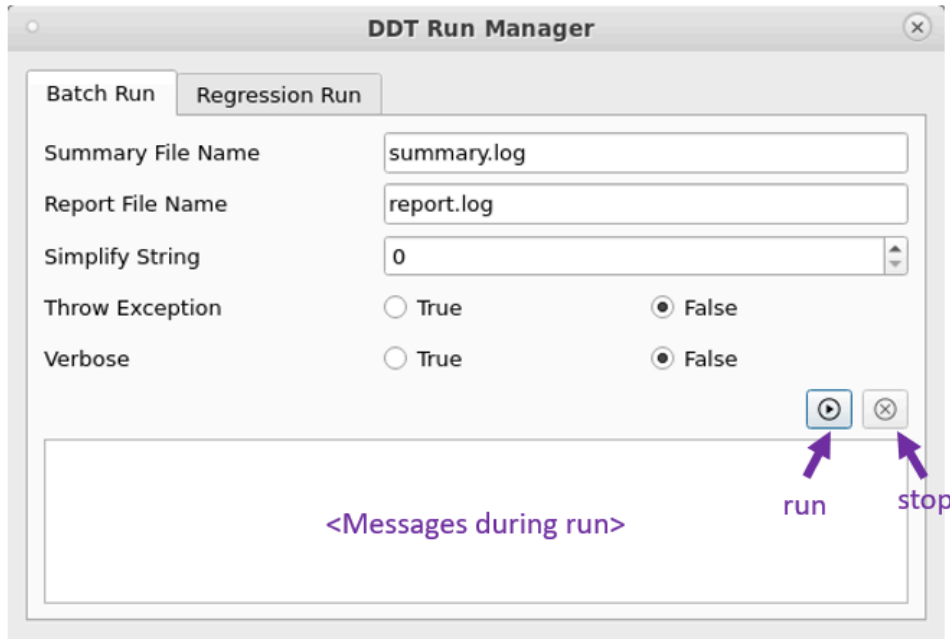
There are several runtime options can be set on `ddt run`:

<code>--sumary <file name></code>	set summary file name for each runtime configuration
<code>--report <file name></code>	like <code>--sumary</code> , set report file name
<code>--target <folder></code>	put all the generated files into the specify folder. use current directory by default
<code>--global-var <file name></code>	Redefine global variable with values specified in the file
<code>--throw</code>	exception will be thrown rather than ignored during execution

It is worth noting that DDT provides runtime option `--global-var` to allow user changing global variable values during `ddt run`. This capability lets you running the same session with different waveform files or logs without updating session.

Run on GUI

You can run a DDT session from GUI as well. By clicking 'Run Manager' () button, DDT will pop out a dialogue as shown in below figure:



In this dialogue we can change default settings in command ``ddt run``, such as summary file name, report file name, and so on.

Note:

Do not forget to save session before batch run, and any changes without saved will not be applied on batch run.

Regression Run

Regression run consists of the following sections:

Run via Command

Just like the use of batch run, DDT supports command for regression run: ``ddt regrun <session file>``. For example, `%> ddt regrun <session_file>`

There are several runtime options can be set for regrun:

<code>--sumary <file name></code>	set summary file name for each runtime configuration
<code>--report <file name></code>	like <code>--sumary</code> , set report file name

--target <folder>	put all the generated files into the specify folder. use current directory by default
--jobs <number>	Specify number of workers for running regression run configurations
--record-file <file name>	Use the given file instead of the regression run configuration stored in the session file.

Run on GUI

In the *Run Manager* dialogue, clicking tab *Regression Run* can enter the configurations regression run. With the help of regression run manager, you can define a set of runtime configurations, and then DDT can run all or selected configurations sequentially or concurrently.

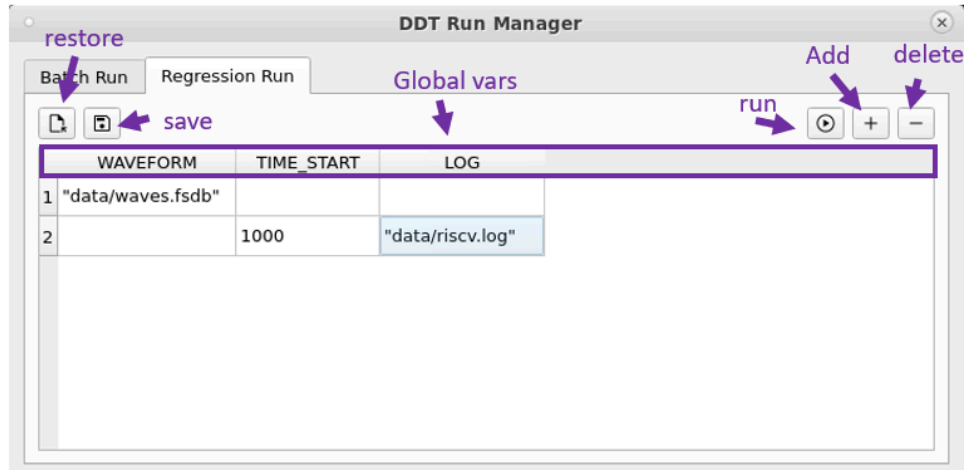
You can find an empty table in regression run tab that is waiting for users to fill in runtime configurations. Columns of the table consists of the names of defined global variables, and it is changed with respect to the defined global variables. Thus, any add/remove/update or even undo/redo operations on global variables changes columns of this table.

Based on the columns, you can use $+$ or $-$ to add a new row or delete selected rows, where one row represents one runtime configuration. To select a row, just click the index cell and the selected rows are highlighted. In each row, you can then fill in new values for every global variable.

For example, the following figure illustrates a table containing two runtime configurations (two rows), where the first configuration sets ``WAVEFORM`` to a new value `"data/waves.fsdb"`, and the second configuration sets ``TIME_START`` and ``LOG`` to 1000 and `"data/riscv.log"` respectively.

You can leave the cells empty if their values are not required to be changed. After finishing the settings of runtime configurations, click *Run* (⏮) button to start the process of regression run. In this example, DDT runs the two configurations defined in the table.

For the convenience of modifying or creating configurations from daily regression, you can save or restore regression run configurations to/from files via the buttons  and . The saved regression run configurations can then be used via command line usage.



Organize Run Report

You can generate organized reports based on the file *summary.log*. The usage can be found by `ddt summary --help` command.

For example,

```
%> ddt summary --help
usage: Process execution from command line. summary [-h] [--output
OUTPUT] [--format FORMAT] [--listall]
summary_file
positional arguments:
summary_file specify summary file
optional arguments:
-h, --help show this help message and exit
--output OUTPUT specify output file
--format FORMAT specify convert format (table, report,
log, json)
--listall list all nodes even if it is not executed
```

Note:

The format 'table' means the CSV format. The format 'json' means the JSON format.

```
%> ddt summary summary.log --format csv --output dvfs_ddt_csv.log
%> cat dvfs_ddt_csv.log
```

```
uid,name,level,parent,child,status
1,Test Fail,0,[0],[2, 3],TRUE
2,UVM Error,1,[1],[],TRUE
3,AXI4S_PSTATE_MISCOMP_STS,1,[1],[4, 5],TRUE
4,PSTATE_TIMEOUT_RACK_THROTTLE_START,2,[3],[],FALSE
5,previous_pchannel_cstate_exit_complete,2,[3],[8],TRUE
8,next_any_throttle_or_no_throttle_message_expected,3,[5],[9],TRUE
9,1st UVM Error,4,[8],[10],TRUE
10,next_any_throttle_or_no_throttle_message_expected,5,[9],[],TRUE
```

The **report** format appears similar to a tree.

For example,

```
%> ddt summary summary.log --format report --output dvfs_ddt_report.log
%> cat dvfs_ddt_report.log
```

```
Test Fail: TRUE [1]
  UVM Error: TRUE [2]
    AXI4S_PSTATE_MISCOMP_STS: TRUE [3]
      PSTATE_TIMEOUT_RACK_THROTTLE_START: FALSE [4]
        previous_pchannel_cstate_exit_complete: TRUE [5]
          next_any_throttle_or_no_throttle_message_expected: TRUE [8]
            1st UVM Error: TRUE [9]
              next_any_throttle_or_no_throttle_message_expected: TRUE [10]
```

The format **log** shows the inputs/outputs of each node during the DDT evaluation as following:

```
%> ddt summary summary.log --format log --output dvfs_ddt_report.log
%> cat dvfs_ddt_report.log
```

```
1) node name = Test Fail:
  result = TRUE
  input = {File name: "/global/cfsdid/jackey/work/caps/dvfs.log", regex: "/TESTCASE_FAILED/"}
  output = {result: True, time: 0, severity: VERDIERROR, dvfs_data: , reporter: , reporter: , scope: , time: 0, time_start: 0, time_end: 0, time_delta: 0, number: 1200}
  message = {}

2) node name = UVM Error:
  result = TRUE
  input = {File name: "/global/cfsdid/jackey/work/caps/dvfs.log", regex: "/UVM_ERROR/"}
  output = {result: True, time: 0, severity: VERDIERROR, dvfs_data: , reporter: , reporter: , scope: , time: 0, time_start: 0, time_end: 0, time_delta: 0, number: 1000}
  message = {}

3) node name = AXI4S_PSTATE_MISCOMP_STS:
  result = TRUE
  input = {File name: "/global/cfsdid/jackey/work/caps/dvfs.log", regex: "/AXI4S_PSTATE_MISCOMP_STS/"}
  output = {result: True, time: 0, severity: VERDIERROR, dvfs_data: , reporter: , reporter: , scope: , time: 0, time_start: 0, time_end: 0, time_delta: 0, number: 2000}
  message = {}

4) node name = PSTATE_TIMEOUT_RACK_THROTTLE_START:
  result = FALSE
  input = {File name: "/global/cfsdid/jackey/work/caps/dvfs.log", regex: "/PSTATE_TIMEOUT_RACK_THROTTLE_START/"}
  output = {result: False, time: 0, severity: VERDIERROR, dvfs_data: , reporter: , reporter: , scope: , time: 0, time_start: 0, time_end: 0, time_delta: 0, number: 100}
  message = {}

5) node name = previous_pchannel_cstate_exit_complete:
  result = TRUE
  input = {File name: "/global/cfsdid/jackey/work/caps/dvfs.log", regex: "/PREVIOUS_PCHANNEL_CSTATE_EXIT_COMPLETE/"}
  output = {result: True, time: 0, severity: VERDIERROR, dvfs_data: , reporter: , reporter: , scope: , time: 0, time_start: 0, time_end: 0, time_delta: 0, number: 100}
  message = {}

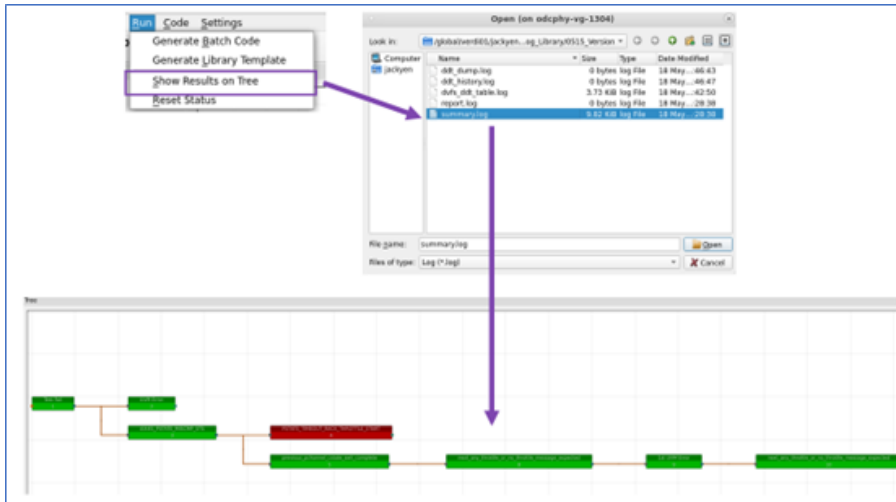
8) node name = next_any_throttle_or_no_throttle_message_expected:
  result = TRUE
  input = {File name: "/global/cfsdid/jackey/work/caps/dvfs.log", regex: "/NEXT_ANY_THROTTLE_OR_NO_THROTTLE_MESSAGE_EXPECTED/"}
  output = {result: True, time: 0, severity: VERDIERROR, dvfs_data: , reporter: , reporter: , scope: , time: 0, time_start: 0, time_end: 0, time_delta: 0, number: 100}
  message = {}

9) node name = 1st UVM Error:
  result = TRUE
  input = {File name: "/global/cfsdid/jackey/work/caps/dvfs.log", regex: "/1ST_UVM_ERROR/"}
  output = {result: True, time: 0, severity: VERDIERROR, dvfs_data: , reporter: , reporter: , scope: , time: 0, time_start: 0, time_end: 0, time_delta: 0, number: 100}
  message = {}

10) node name = next_any_throttle_or_no_throttle_message_expected:
  result = TRUE
  input = {File name: "/global/cfsdid/jackey/work/caps/dvfs.log", regex: "/NEXT_ANY_THROTTLE_OR_NO_THROTTLE_MESSAGE_EXPECTED/"}
  output = {result: True, time: 0, severity: VERDIERROR, dvfs_data: , reporter: , reporter: , scope: , time: 0, time_start: 0, time_end: 0, time_delta: 0, number: 100}
  message = {}
```

The **summary.log** can be loaded back to the DDT GUI through the following steps:

1. Click **Run > Show Results on Tree**
2. Select **summary.log**, the DDT execution result is shown as shown in the following illustration:



The green node is the node which is PASS, the red node is the node which is FAIL, and the gray node is the node which is NOT RUN.

Note:

Use the menu **Run > Reset Status**, to reset the DDT to the state of not-yet-execution.

Debug Decision Tree Library

Debug Decision Tree (DDT) offers several handy libraries for you to create the decision tree to capture the debug process. The libraries can be found in the library pane of DDT GUI. For more details, refer to [Debug Decision Tree Usage](#).

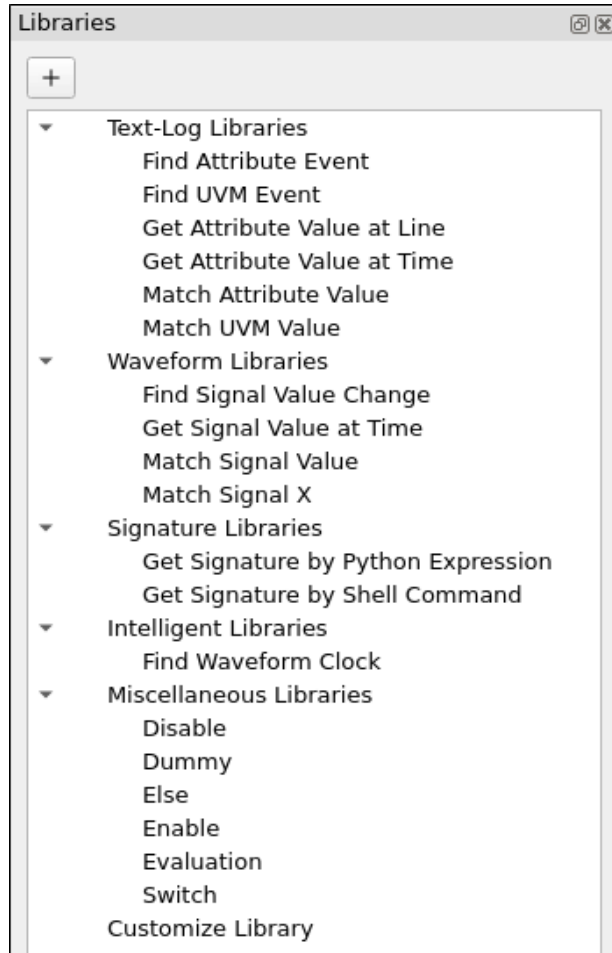
A tree node is created by a pre-defined or Customize Library. You can config input arguments of a node, such as source file name, searching window, matching condition, and so on.

Dependent nodes are connected to form a tree. A **packet** is transmitted from parent to children, such that a child node could utilize information from its parent.

A node would first **Decide** to run or not, according to certain condition of parent and arguments. If true, then **Execute** to produce its output packet, such as finding certain pattern in log or matching signal value in the waveform.

There are [Text-Log Libraries](#), [Waveform Libraries](#), [Signature Libraries](#), [Miscellaneous Libraries](#), and [Customize Library](#) that belong to the tree library.

The following are the introduction to some of the arguments for these libraries:



In this section, let us go through the decision logic of each library.

Specify Attributes and Signals

Decision logic of Specify Attributes and Signals library contains the following sections:

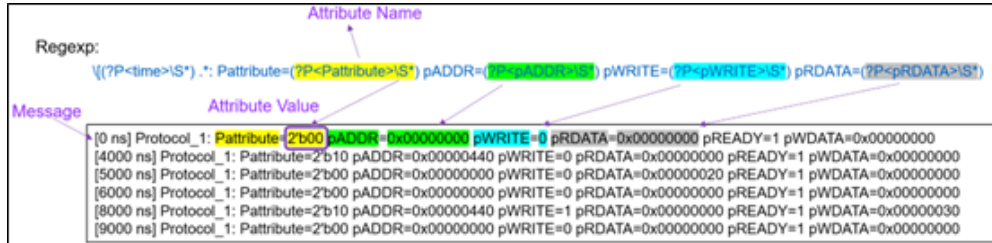
Textlog Attributes

For textlog related libraries, attributes are specified with the `regex` in two types: `str` or `dict`.

In a simulation text log, a **log-line** is the texts of a line in a log. For each log-line, it could contain several **attribute** values. Users can use regular expression (`regex`) to extract attributes values from log-lines, also give a name for each attribute.

The regular expression can be written using named-group syntax where the attributes are implicitly specified in the pattern. For example, the following regex defines the named group *time*, *Pattribute*, *pADDR*, *pWRITE*, and *pRDATA*. The value of the group *time* in the regex will be used for the time in a message.

```
r"\"[(?P<time>\\S*) .*: Pattribute=(?P<Pattribute>\\S*)
pADDR=(?P<pADDR>\\S*) pWRITE=(?P<pWRITE>\\S*) pRDATA=(?P<pRDATA>\\S*) "
```



Named-group is extremely useful when multiple expressions are put together where different fields could be mapped to the same attribute. Use the syntax `|` to concatenate multiple regex string. Below is an example to specify multiple regex with named-group.

```
regex = '|'.join([
    r"s*(?P<time>\\S+)\\.\\.ns.*PSTATE
    (?P<throttle_type>\\S+)_THROTTLE_START message expected.*",
    r"s*(?P<time>\\S+)\\.\\.ns.*PSTATE (?P<throttle_type>)NO_THROTTLE
    message expected.*"
])
```

Without named-group syntax, the default attribute names are numbered from 1 to k, where k is the number of groups associated with the regular expression pattern.

Attribute names can be explicitly specified by a dict where the key is the pattern, and the value is a list of attribute names. In this case, the length of the list must be the same as the number of groups.

Waveform Signals

For waveform related libraries, signals can be specified in three types: str, list, or SignalMapping.

In a simulation waveform, a **signal** contains several **value-changes**.

A signal is specified in its full name with scope separated by `'.'`, such as `"top.cpu.clk"`.

A list of signal full names is also accepted, such as `["top.cpu.clk", "top.cpu.rst"]`.

Signal Mappings is a format produced by Signal Mapping Wizard, which is a tab named "Signal Mappings" and located in Global Variables. In a Signal Mapping, the keys are simple names or aliases, and the values are signals' full names, e.g. `{"clk": "top.cpu.clk", "rst": "top.cpu.rst"}`.

It is recommended to use Signal Mapping for better readability and maintainability.

The `value` in the returned `Packet` shows values corresponding to the signals that are chosen in `signal_names`. Notice the Signal Mapping keys are simple names or aliases for ease of access.

Search Range

Search Range contains the following sections:

Line Range

For textlog related libraries, the start of line range `line_start` is default to 0, where the line range starts from the beginning or the end of the log, according to `search_forward`. It ends at `line_start +/- line_delta` where the end line is not included.

Time Range

For textlog related libraries, the time range applied if 'time' is in the attributes, but it is always applied in waveform related libraries. Time is assumed to be monotonic increasing in the log or waveform. The start of time range `time_start` is default to None, where the time range starts from the beginning or the end of the log or waveform, according to `search_forward`.

The ends at `time_start +/- time_delta` where the end time is not included. The `exclude_time_start` argument means if the search start from the same time of `time_start`. If it is False, the search will start from `time_start +/- 1`, according to `search_forward`.

For waveform related libraries, time can be specified in two formats: `int` or `str`. When an integer is given, the time-unit of the waveform is used. When a time-with-unit string is given, it is converted to the time-unit of the waveform. For example, '1us' is converted to 1000 when the time-unit of the waveform is 'ns'.

Value Compare

The `values` to compare can be specified in three types: `str`, `list` or `dict`.

For a single value as `str`, it is broadcasted and compared with all items. For a `list` of values, its length should be matched with the number of items in `regex` or `signal_names`. They are zipped to be compared one-by-one. A `dict` of signal-value mappings can compare a subset of items.

When there are more than one attribute to be compared, `all_match` determines the overall behavior. If it is set to True, all attributes must match the given values. Otherwise, any attribute matches the given value is treated as successful.

Textlog Value Compare

For textlog related libraries, the value format is text string, as extract by regular expressions by default. If the given right-hand-side value is in another type, then the attribute is implicitly converted this type.

Assume the given regular expression contains two attributes *pstate* and *pcode*, in addition to *time*.

```
r"\s*(?P<time>\S+).*PERF_P(?P<pstate>\S+).*trigger (?P<pcode>.*)_PSTATE"
```

Take a single value "43" as an example, all attributes are compared with the string value "43".

For a list of values ["31", "CSTATE"], the attribute *pstate* is compared with "31" and *pcode* is compared with "CSTATE".

A dict of signal-value mappings {'pcode': "CSTATE"} means the attribute *pcode* is compared with "CSTATE".

Waveform Value Compare

For waveform related libraries, the value format is binary string with x/z, example, "100x" by default. Users can explicitly specify the radix as binary, octal, decimal, or hexadecimal with prefix "0b", "0o", "0d", or "0h", respectively. For example, "0hc" and "0d10" both equivalent to the binary string "1010". The value can be an integer as well, which is converted to a binary string format implicitly.

Assume two 8-bit signals are given: *addr* and *data*.

Take a single value "10101111" as an example, all signals are compared with the binary value "10101111".

For a list of values ["0haf", 127], *addr* is compared with "10101111" and *data* is compared with "11111111".

A dict of signal-value mappings {'data': "0d127"} means **data** is compared with "11111111".

Note:

The key should be simple-name or alias when `signal_names` is specified as a dict.

Enum

Enum contains the following sections:

UVM Severity

```
class UVMSeverity(value)
    UVM_Severity

    UVM_FATAL = 1
    UVM_ERROR = 2
    UVM_WARNING = 3
    UVM_INFO = 4
```

Text-Log Libraries

DDT text-Log libraries allow users to get, find, and match attribute values of a log.

To set up text-log libraries, you may need to know [Textlog Attributes](#). In Find and Match libraries, the search range is determined by [Line Range](#) and [Time Range](#). In Match libraries, user have to set up [Textlog Value Compare](#). For UVM log related libraries, there is an argument [UVM Severity](#).

Enum

Enum of Text-Log libraries contains the followings sections:

Compare Operator

```
class CompareOperator(value)
    Compare Operator
```

EQUAL

Attribute is equal to the given value.

NOT_EQUAL

Attribute is not equal to the given value.

GREATER

Attribute is greater than the given value.

NOT_GREATER

Attribute is not greater than (less than or equal to) the given value.

LESS

Attribute is less than the given value.

NOT_LESS

Attribute is not less than (greater than or equal to) the given value.

IN_LIST

Attribute is in the given list of values.

CONTAIN

Attribute contains a sub-string or matches a regular expression of the given value.

UVM Severity

```
class UVMSeverity (value)
    UVM Severity

    UVM_FATAL = 1
    UVM_ERROR = 2
    UVM_WARNING = 3
    UVM_INFO = 4
```

Library

The following table summarizes available libraries and associated descriptions for text log:

Library Name	Brief Description
Find Attribute Event	Find a message in a log containing attribute events specified by the regular expression.
Find UVM Event	Find a message in a log containing UVM attribute events, such as severity, reporter, and code.
Get Attribute Value at Line	Get attribute values at certain line.
Get Attribute Value at Time	Get attribute values at certain time.
Match Attribute Value	Match attribute event value for groups.
Match UVM Value	Match UVM event values.

Find Attribute Event

Description:

Find a message in a log containing attribute events specified by the regular expression.

Arguments:

- `file_name (str)` – Log file name.
- `regex (Union[str, Mapping[str, List[str]]])` – Regular expression to parse log.
- `line_start (int)` – Start line to search.
- `line_delta (Optional[int])` – Delta search limit regarding `line_start`.
- `time_start (int)` – Start time to search.
- `time_delta (Optional[int])` – Delta search limit regarding `time_start`.
- `exclude_time_start (bool)` – Exclude transition occurring at `time_start`.
- `search_forward (bool)` – Search forward or backward.
- `event_count (int)` – Number of event transition counts.
- `any_event (bool)` – Count transition if any attribute has event.

Output:

Returns attribute values and line for an event.

- `line_content: str = ''`
- `line_number: int = 0`
- `result: object = False`
- `values: Mapping[str, object]`

Find UVM Event

Description:

Find a message in a log containing UVM attribute events, such as severity, reporter, and code.

Arguments:

- `file_name (str)` – Log file name.
- `line_start (int)` – Start line to search.
- `line_delta (Optional[int])` – Delta search limit regarding `line_start`.
- `time_start (int)` – Start time to search.
- `time_delta (Optional[int])` – Delta search limit regarding `time_start`.

- **exclude_time_start** (bool) – Exclude transition occurring at `time_start`.
- **severity** (Optional[UVMSeverity]) – UVM Severity.
- **search_forward** (bool) – Search forward or backward.
- **event_count** (int) – Number of event transition counts.

Output:

Returns attribute values and line for a UVM event.

- **code:** `str = ''`
- **line_content:** `str = ''`
- **line_number:** `int = 0`
- **message:** `str = ''`
- **reporter:** `str = ''`
- **result:** `object = False`
- **severity:** `Optional[UVMSeverity] = None`
- **time:** `int = -1`

Get Attribute Value at Line

Description:

Get attribute values at certain line.

Arguments:

- **file_name** (str) – Log file name.
- **regex** (Union[str, Mapping[str, List[str]]]) – Regular expression to parse log.
- **line_number** (int) – Line number.

Output:

Returns attribute values at line.

- **line_content:** `str = ''`
- **line_number:** `int = 0`
- **result:** `object = False`
- **values:** `Mapping[str, object]`

Get Attribute Value at Time

Description:

Get attribute values at certain time.

Arguments:

- `file_name` (`str`) – Log file name.
- `regex` (`Union[str, Mapping[str, List[str]]]`) – Regular expression to parse log.
- `time` (`int`) – Snapshot time to retrieve values.

Output:

Returns attribute values at line.

- `line_content`: `str` = ''
- `line_number`: `int` = 0
- `result`: `object` = `False`
- `values`: `Mapping[str, object]`

Match Attribute Value

Description:

Match attribute event value for groups.

Arguments:

- `file_name` (`str`) – Log file name.
- `regex` (`Union[str, Mapping[str, List[str]]]`) – Regular expression to parse log.
- `values` (`object`) – Group values to match.
- `compare` (`CompareOperator`) – Compare operator to match group values.
- `all_match` (`bool`) – Return `True` if found matched values for all groups.
- `line_start` (`int`) – Start line to search.
- `line_delta` (`Optional[int]`) – Delta search limit regarding `line_start`.
- `time_start` (`int`) – Start time to search.
- `time_delta` (`Optional[int]`) – Delta search limit regarding `time_start`.

- `exclude_time_start` (bool) – Exclude event occurring at `time_start`.
- `search_forward` (bool) – Search forward or backward.
- `event_count` (int) – Number of event counts.

Output:

Returns matched attribute values and line for an event.

- `line_content`: *str* = ''
- `line_number`: *int* = 0
- `result`: *object* = *False*
- `values`: *Mapping[str, object]*

Match UVM Value

Description:

Match UVM event values.

Arguments:

- `file_name` (str) – Log file name.
- `severity` (Optional[UVMSeverity]) – UVM severity to match.
- `reporter` (Optional[str]) – UVM reporter to match.
- `code` (Optional[str]) – UVM code to match.
- `message` (Optional[str]) – UVM message to match..
- `compare` (CompareOperator) – Compare operator to match group values.
- `line_start` (int) – Start line to search..
- `line_delta` (Optional[int]) – Delta search limit regarding `line_start`.
- `time_start` (int) – Start time to search.
- `time_delta` (Optional[int]) – Delta search limit regarding `time_start`.
- `exclude_time_start` (bool) – Exclude event occurring at `time_start`.
- `search_forward` (bool) – Search forward or backward..
- `event_count` (int) – Number of event counts.

Output:

Returns matched UVM fields and line for a UVM event.

- `code: str = ''`
- `line_content: str = ''`
- `line_number: int = 0`
- `message: str = ''`
- `reporter: str = ''`
- `result: object = False`
- `severity: Optional[UVMSeverity] = None`
- `time: int = -1`

Waveform Libraries

DDT waveform libraries allows you to get, find, and match values or times of one or more signals.

To set up waveform libraries, you may need to know [Waveform Signals](#). In Find and Match libraries, the search range is determined by [Time Range](#). In Match libraries, user must set up [Waveform Value Compare](#).

Enum

Enum of Waveform libraries contains the following sections:

Compare Operator

```
class CompareOperator(value)
```

Compare Operator

EQUAL

Attribute is equal to the given value.

NOT_EQUAL

Attribute is not equal to the given value.

GREATER

Attribute is greater than the given value.

NOT_GREATER

Attribute is not greater than (less than or equal to) the given value.

LESS

Attribute is less than the given value.

NOT_LESS

Attribute is not less than (greater than or equal to) the given value.

IN_LIST

Attribute is in the given list of values.

CONTAIN

Attribute contains a sub-string or matches a regular expression of the given value.

Library

The following table summarizes available libraries and associated descriptions for waveform:

Library Name	Brief Description
Find Signal Value Change	Find transition time and value for signals.
Get Signal Value at Time	Get value of signal(s) at certain time.
Match Signal Value	Find time to match signal values.
Match Signal X	Find time to match signal having X values.

Find Signal Value Change

Description:

Find transition time and value for signals.

Arguments:

- `file_name` (`str`) – Waveform file name.
- `signal_names` (`Union[str, List[str], SignalMapping]`) – Signal full name, list of signal names, or Signal Mapping.
- `time_start` (`Optional[Union[int, str]]`) – Start time to search.
- `time_delta` (`Optional[Union[int, str]]`) – Delta search limit regarding `time_start`.
- `exclude_time_start` (`bool`) – Exclude value-change occurring at `time_start`.
- `search_forward` (`bool`) – Search forward or backward along timeline..

- **value_change_count** (int) – Number of value-change transition counts.
- **any_change** (bool) – Count transition if any signal has value-change.

Output:

Returns found value-change time and values of `signal_names` starting from `time_start`.

- **result:** *object = False*
- **time:** *int = -1*
- **values:** *Mapping[str, str] = None*

Get Signal Value at Time

Description:

Get value of signal(s) at certain time.

Arguments:

Returns values of `signal_names` at time.

- **file_name** (str) – Waveform file name.
- **signal_names** (Union[str, List[str], SignalMapping]) – Signal full name, list of signal names, or Signal Mapping.
- **time** (Union[int, str]) – Snapshot time to retrieve values.

Output:

Returns values of `signal_names` at time.

- **result:** *object = False*
- **time:** *int = -1*
- **values:** *Mapping[str, str] = None*

Match Signal Value

Description:

Find time to match signal values.

Arguments:

- **file_name** (str) – Waveform file name.
- **signal_names** (Union[str, List[str], SignalMapping]) – Signal full name, list of signal names, or Signal Mapping.
- **values** (Union[Mapping[str, str], List[str], str]) – Signal values to match.

- **compare** (`CompareOperator`) – Compare operator to match signal values.
- **all_match** (`bool`) – Return `True` if found matched values for all signals.
- **time_start** (`Optional[Union[int, str]]`) – Start time to search.
- **time_delta** (`Optional[Union[int, str]]`) – Delta search limit regarding `time_start`.
- **exclude_time_start** (`bool`) – Exclude transition occurring at `time_start`.
- **search_forward** (`bool`) – Search forward or backward along timeline.

Output:

Returns values of `signal_names` starting from `time_start`.

- **result:** `object = False`
- **time:** `int = -1`
- **values:** `Mapping[str, str] = None`

Match Signal X

Description:

Find time to match signal having X values.

Arguments:

- **file_name** (`str`) – Waveform file name.
- **signal_names** (`Union[str, List[str], SignalMapping]`) – Signal full name, list of signal names, or Signal Mapping.
- **all_match** (`bool`) – Return `True` if found matched values for all signals.
- **time_start** (`Optional[Union[int, str]]`) – Start time to search.
- **time_delta** (`Optional[Union[int, str]]`) – Delta search limit regarding `time_start`.
- **exclude_time_start** (`bool`) – Exclude transition occurring at `time_start`.
- **search_forward** (`bool`) – Search forward or backward along timeline.

Output:

Returns values of `signal_names` starting from `time_start`.

- **result:** *object* = *False*
- **time:** *int* = *-1*
- **values:** *Mapping[str, str]* = *None*

Signature Libraries

DDT signature libraries allow users to extract the signatures by several ways. A signature is a special string that is known for a specific feature. For example, if a test case runs with SVA, a signature can be the scope where the SVA errors happened.

To set up textlog and waveform related libraries, you may need to know [Specify Attributes and Signals](#). In Find and Match libraries, the search range is determined by [Line Range](#) and [Time Range](#). In Match libraries, user have to set up [Value Compare](#). For UVM-log related libraries, there is an argument [UVM Severity](#).

Weight

There is an `int` type argument `weight` default to 1 using for bucketing for each node. It refers to assigning different levels of importance to signatures during the bucketing process and allows user to prioritize certain signatures over others based on their significance.

By assigning weights, we can control the influence of specific signatures on the bucketing results, resulting in more meaningful and tailored buckets. For some nodes whose signatures will not be used in the bucketing, the weight can be set to 0.

Encode Option Usage

The `encode_option` default to `categorical` is used for data preprocessing. In some situation, the datatype could not be inferred implicitly.

For example, in the case of [Get Signature by Shell Command](#), the output of the command execution is always a string, However, some signatures want to be set to a numerical type.

For those signatures we cannot get the types, users must define the encode option for them. Typically, integers and floating-point numbers are numerical signatures, and others are categorical signatures.

Enum

```
class EncodeOption (value)
```

Encode Option

```
CATEGORICAL = 1  
NUMERICAL = 2
```

Library

The following table summarizes available libraries and associated descriptions for signature extraction:

Library Name	Brief Description
Find Log Attribute Signature	Extract signature in a log containing attribute events specified by the regular expression.
Find Log UVM Signature	Extract signature values from messages of a UVM log.
Find Waveform Value Change Signature	Find value change of signal(s) by certain time range.
Get Signature by Python Expression	Get signature by Python Expression
Get Signature by Shell Command	Extract signatures using UNIX shell command(s)
Get Waveform Value Signature	Get value of signal(s) at certain time.

Find Log Attribute Signature

Description:

Extract signature in a log containing attribute events specified by the regular expression.

Arguments:

- `file_name` (str) – Log file name.
- `regex` (Union[str, Mapping[str, List[str]]]) – Regular expression to parse log.
- `line_start` (int) – Start line to search.
- `line_delta` (Optional[int]) – Delta search limit regarding `line_start`.
- `time_start` (Optional[int]) – Start time to search.
- `time_delta` (Optional[int]) – Delta search limit regarding `time_start`.
- `exclude_time_start` (bool) – Exclude transition occurring at `time_start`.
- `search_forward` (bool) – Search forward or backward.
- `event_count` (int) – Number of event transition counts.
- `any_event` (bool) – Count transition if any attribute has event.

- **weight** (int) – Signature weight used for bucketing.
- **encode_option** (EncodeOption) – Signature type in bucketing: CATEGORICAL or NUMERICAL.

Output:

Return attributes.

- **result:** *object = False*
- **signature:** *Mapping[str, Union[str, int, Mapping[str, object]]]*
- **weight:** *int = 0*

Find Log UVM Signature

Description:

Extract signature values from messages of a UVM log. Find a message in a log containing UVM attribute events, such as severity, reporter, and code.

Arguments:

- **file_name** (str) – Log file name.
- **line_start** (int) – Start line to search.
- **line_delta** (Optional[int]) – Delta search limit regarding **line_start**.
- **time_start** (Optional[int]) – Start time to search.
- **time_delta** (Optional[int]) – Delta search limit regarding **time_start**.
- **exclude_time_start** (bool) – Exclude transition occurring at **time_start**.
- **severity** (Optional[UVMSeverity]) – UVM Severity.
- **search_forward** (bool) – Search forward or backward.
- **event_count** (int) – Number of event transition counts.
- **weight** (int) – Signature weight used for bucketing.

Output:

Returns weight and signature including time, severity, reporter, code, message, line content, and line number.

- **result:** *object = False*
- **signature:** *Mapping[str, Union[str, int, Mapping[str, object]]]*
- **weight:** *int = 0*

Find Waveform Value Change Signature

Description:

Find value change of signal(s) by certain time range. All the signals are expanded into signature['values'] dictionary without undefined signals. Returns found value-change time and values of `signal_names` starting from `time_start`.

Arguments:

- `file_name` (`str`) – Waveform file name.
- `signal_names` (`Union[str, List[str], SignalMapping]`) – Signal full name, list of signal names, or Signal Mapping.
- `time_start` (`Optional[int]`) – Start time to search.
- `time_delta` (`Optional[int]`) – Delta search limit regarding `time_start`.
- `exclude_time_start` (`bool`) – Exclude transition occurring at `time_start`.
- `search_forward` (`bool`) – Search forward or backward along timeline..
- `value_change_count` (`int`) – Number of value-change transition counts.
- `any_change` (`bool`) – Count transition if any signal has value-change.
- `weight` (`int`) – Signature weight used for bucketing.
- `encode_option` (`EncodeOption`) – Signature type in bucketing: CATEGORICAL or NUMERICAL.

Output:

Returns weight and signature including values, line content, and line number.

- `result`: `object = False`
- `signature`: `Mapping[str, Union[int, Mapping[str, Union[str, int]]]]`
- `weight`: `int = 0`

Get Signature by Python Expression

Description:

Evaluate one-line python expression to extract the signature.

There are some examples. First, the python expression `"2 * 8 + 4"` is to give the signature a constant. The python expression `"packet['signature'] * 100"` is to get the packet from the parent node and do some calculation as a new signature. And this python expression `"int(packet['signature'])"` is to get the packet from the parent node, and change the type of parent's signature to a new signature.

.

Arguments:

- **python_expression (str)** – One-line python expression.

Note:

The `python_expression` is a Python string, hence the double quote must be used for the `python_expression`.

- **weight (int)** – Signature weight used for bucketing.

Output:

Returns weight and signature.

- **result:** `object = False`
- **signature:** `str = ''`
- **weight:** `int = 0`

Get Signature by Shell Command

Description:

Extract signatures using UNIX shell command(s)

For example, the following UNIX command is used to extract the signature string by the command `grep` and `cut` for the log file `test.rpt`:

```
grep 'TLM' test.rpt | cut -d : -f
```

Any script or program that can be run in the UNIX system can be used in command-arguments. Therefore, you can use Python, Perl, C-shell, or other kinds of script to extract the signature.

Arguments:

- **python_expression (str)** – One-line python expression.

Note:

The `python_expression` is a Python string, hence the double quote must be used for the `python_expression`.

- **command (Union[str, List[str]])** – UNIX Command. Notice that the command is a Python string, hence the double quote must be used for the UNIX command.
- **timeout (int)** – Timeout setting for Unix process, a value in seconds that specifies how long to wait for the sub process to complete before timing out.

- **weight** (*int*) – Signature weight used for bucketing.
- **encode_option** (*EncodeOption*) – Signature type in bucketing: CATEGORICAL or NUMERICAL.

Output:

Returns *return_code*, *weight*, and *signature*.

- **result:** *object* = *False*
- **return_code:** *Union[int, List[int]]* = *0*
- **signature:** *str* = ''
- **weight:** *int* = *0*

Get Waveform Value Signature

Description:

Get value of signal(s) at certain time.

All the signals are expanded into *signature['values']* dictionary without undefined signals.

Arguments:

- **file_name** (*str*) – Waveform file name.
- **signal_names** (*Union[str, List[str], SignalMapping]*) – Signal full name, list of signal names, or Signal Mapping.
- **time** (*Union[int, str]*) – Snapshot time to retrieve values.
- **weight** (*int*) – Signature weight used for bucketing.
- **encode_option** (*EncodeOption*) – Signature type in bucketing: CATEGORICAL or NUMERICAL.

Output:

Returns *weight* and *signature* including *time* and *values*.

- **result:** *object* = *False*
- **signature:** *Mapping[str, Union[int, Mapping[str, Union[str, int]]]]*
- **weight:** *int* = *0*

Miscellaneous Libraries

Miscellaneous Libraries contains the following sections:

Library

The following table summarizes available libraries and associated descriptions of miscellaneous usage:

Library Name	Brief Description
Enable	Enable sub-tree.
Disable	Disable sub-tree.
Dummy	Dummy tree node.
Else	Run this node when the given condition is not success
Evaluation	Evaluate expression.
Switch	Switch on/off sub-tree.

Enable

Description:

Enable sub-tree.

Arguments:

This library has no arguments.

Output:

Return `True` to enable tree nodes in the descent sub-tree.

- `result: object = False`

Disable

Description:

Disable sub-tree.

Arguments:

This library has no arguments.

Output:

Return `False` to disable tree nodes in the descent sub-tree.

- `result: object = False`

Dummy

Description:

Dummy tree node.

Arguments:

This library has no arguments.

Output:

Return nothing as a dummy tree node.

- `result: object = False`

Else

Description:

Run this node if the packet does not run successfully.

Arguments:

This library has no arguments.

Output:

Return nothing as the else part of the running condition.

- `result: object = False`

Evaluation

Description:

Evaluate expression.

Arguments:

- `expression (str)` - Python expression to be evaluated.

Output:

Return expression evaluation results.

- `result: object = False`

Switch

Description:

Switch on/off sub-tree.

Arguments:

This library only has argument in mode `decide`

- `control (bool)` - True, False, or a bool variable.

Output:

Returns nothing just like Dummy.

- `result: object = False`

Set Variable

Description:

Set variable to a dictionary `variables`.

Arguments:

- `name (str, default: '')` - Variable name.
- `value (Optional[object], default: None)` - Value of the variable to be set.

Output:

Set a variable with name and value.

- `result: object`
- `variables: dict`

Customize Library

Customize Library contains the following sections:

Generate Library Template

To begin the development of customized libraries, the first step is to generate library template. From the menu bar choosing Run -> Generate Library Template, a file browser pops out asking the destination of template library. The generated template looks like the following (python format):

```
from abc import abstractmethod
from dataclasses import dataclass
from verdilearn.ddt import CustomizeLibrary, globalrc
```

```
# TODO: Add other import here if needed
# TODO: add user-defined utility here
def foo():
    pass

# TODO: replace CustomizeLibrary with the hierarchy in the toolbox
class NewHierarchy(CustomizeLibrary):
    def __init__(self, uid: int):
        super().__init__(uid)

    @classmethod
    def register_name(cls):
        # TODO: this name will be displayed on the toolbox, change to a
        UNIQUE name you like
        return 'NewHierarchy'

    @abstractmethod
    def execute(self, packet: dict, *args, **kwargs):
        '''User defined hierarchy'''

# TODO: replace NewLibrary and NewHierarchy to the name you like
# Note: NewHierarchy should be one of the hierarchy in the toolbox
class NewLibrary(NewHierarchy):
    @dataclass
    class Packet:
        result: object
        # add var name & type below that will returned by execute()

    def __init__(self, uid: int):
        super().__init__(uid)

    @classmethod
    def register_name(cls):
        # TODO: this name will be displayed on the toolbox, change to a
        UNIQUE name you like
        return 'NewLibrary'

    def decide(self, packet: dict) -> bool:
        return super().decide(packet)

# TODO: implement library behavior in this method
def execute(self, packet: dict
    # add more parameters here
) -> Packet:
    '''Write docstring for input/output here
    Args:
    Mode:
    '''
    ret = NewLibrary.Packet(False)
    # modify lib name and other default values
    return ret
```

Make Customize Library

There are several lines marking TODO that require user to update.

1. [NewHierarchy and NewLibrary](#)
2. [register_name\(\)](#)
3. [decide\(\)](#)
4. [execute\(\)](#)
5. [Packet](#)

During the implementation of a library, developer may want to know the information of current node, useful utilities and class members are listed in [Useful utils](#). Finally, short example is described in [Example of Customize Library](#) section.

NewHierarchy and NewLibrary

DDT library view uses the class inheritance hierarchy to display. Thus `NewHierarchy` class in the template library becomes a new level under `CustomizeLibrary` in library view because it inherits `CustomizeLibrary` class. Following the same rule, `NewLibrary` becomes the leaf item under `NewHierarchy`.

You can change `NewHierarchy` and `NewLibrary` to whatever name you like or create more hierarchies or libraries via following the same rule.

register_name()

`register_name()` defines the name showing on library view. Unlike the class names `NewHierarchy` or `NewLibrary` that shall follow python naming style, name returned by `register_name()` can be any valid python string. However, DDT does not accept duplicated names to avoid potential confusion. Hierarchy and library names returned by `register_name()` **SHALL** be unique.

decide()

Method `decide()` defines when a node should run according to the return value of its parent node (argument packet). The default behavior of `decide()` is based on the `result` field in packet that DDT activates the execution of a node only if `result` field evaluates to `True`.

execute()

This method defines the main behavior of a node when it is activated. Once complete, `execute()` should return a `Packet` describing the return value of this library. Every argument in `execute()` becomes a configuration on config view.

To give hints to user configuring a node on GUI, library developer can give type hint and short description for every argument. Type hint of an argument allows DDT to list data with

the same type in the combo box, and description detailed in docstring of `execute()` can be divided into 3 parts for different purposes:

1. *Method description*: Comments for library developers to know the behavior of this library.
2. *Args*: Every argument shall be described in format `name: description` in one line, and DDT then scans all these descriptions and presented on GUI.

Note:

All the arguments and their order should be the **same** in the argument list and *Args*.

3. *Mode*: When a powerful library contains many arguments, and not all arguments will be used at the same time, developer can divide arguments to several groups, and each group is formatted as `name: comma separated argument list`. Thereafter, GUI has the same number of radio buttons as the name developer specified in groups. Only arguments specified in the argument list of a group is shown on GUI. This allows user focusing on necessary arguments.

Packet

Execution result of a node contains two parts: execution result and generated data. Execution results shall always be provided via member `result`, which is a reserved member in DDT and **SHALL** not be removed.

Each generated data should provide a name and associated data type. Since the generated data is passed to its descendant nodes, and allow descendant node choosing generated data from combo box, provided data type allow DDT filtering out data with incompatible data type, and shows only parts of possible data in combo box.

Useful Utils

member or method	description
<code>self.uid</code>	get unique ID (type: <code>int</code>) of this node in DDT
<code>self.user_name</code>	get user defined name of this node
<code>self.get_parent()</code>	get a list of parent node. This list contains only 0 or 1 node
<code>self.get_child()</code>	get a list of child node(s)
<code>self.get_packet(uid)</code>	get packet associated with the given <code>uid</code> .
<code>self.report_msg(msg)</code>	use this method instead of <code>print()</code> to dump <code>msg</code> (type: <code>str</code>). Dumped message can be found in <code>ddt_dump.log</code>

self.set_data(key, data)	use this method to store (large) user data associated with the given key
self.get_data(key)	use this method to get user data associated with the given key that set by <code>set_data()</code>
self.has_data(key)	use this method to check if user data associated with key has set

Example of Customize Library

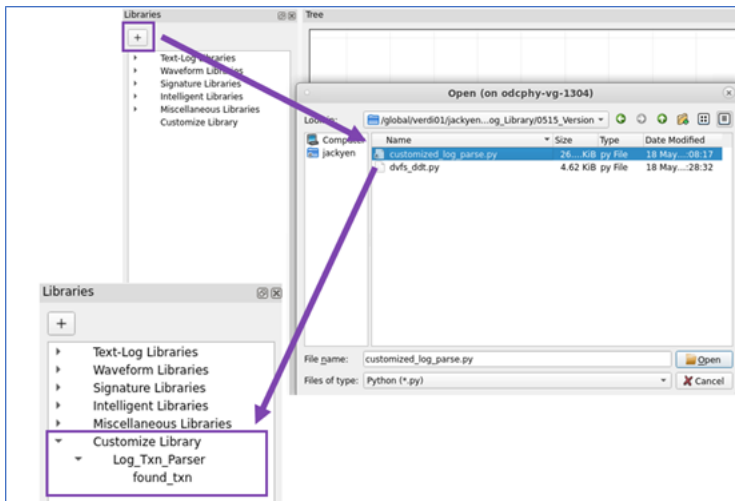
Please refer to our demo case: Customized Library

Import a Customized Library

You can develop a customized library for DDT. An example of the customized DDT library can be found in the following path:

```
$VERDI_HOME/share/NPI/python/demo/ddt/Customized_Lib/  
customized_log_parse.py
```

In the Library widget, use the '+' button to add the customized library. A customized library is shown in the **Libraries Pane** as following:



Note:

The path of the customized library file is recorded to the DDT session. To launch DDT GUI or run the DDT, the session file as well as the customized library is required to be existent.

DDT_WORK_ROOT

By default, DDT session uses the absolute path to record the location of the customized library (.py). It is possible to replace the absolute path with a specific environment variable “**DDT_WORK_ROOT**” for that path. For example, you can modify the DDT session of the aforementioned example to the following:

```
"ddt_global_var": [
  [
    "dvfs_log",
    "\"./Logs/dvfs.log\""
  ],
  [
    "dvfs_scb_trk",
    "\"./Logs/dvfs_scoreboard.trk\""
  ]
],
"ddt_env_var": [],
"ddt_signal_map": [],
"ddt_regex": [],
"ddt_dfop": [],
"ddt_custom_lib_path": [
  "/global/user/jackyen/DDT/custom/customized_log_parse.py"
],
"hash_id": "7d71fb1b4330ec3e652740d139f364a43d714047"
```

By setting the environment variable “**DDT_WORK_ROOT**”, you can organize the path of the customized libraries to run DDT.

Batch Mode

DDT supports batch mode for operating DDT via commands.

```
%> ddt batch &
```

An interactive shell gets prompt and wait for you to enter the commands as following:

```
*****
*                                     *
*                               Warning *
*                                     *
* This is a RD version and is not for formal release. *
* For testing purposes only.           *
*                                     *
*****
>>> █
```

You can enter the DDT commands and a valid Python code in the interactive shell. Available DDT commands can be queried via typing `help()` , or via the documentation provided in section 6.

```

help on ProgramInterface in module verdilearn.ddt_backend object:

class ProgramInterface(globalrc._DDTBaseObject)
|
| DDT API
|
| Method resolution order:
|   ProgramInterface
|   globalrc._DDTBaseObject
|   builtins.object
|
| Methods defined here:
|
|   __getattr__(self, name)
|       Re-implement __getattr__ to record function call automatically
|
|   __init__(self, disable_web=False, license: str=None)
|
|   add_customized_lib(self, filename: str)
|       add customized library
|
|       Args:
|         filename: file name of the customized library
|
|   add_global_var_field(self, tab_type: globalrc.GlobalVarTabType) -> 'int'
|       add an empty field in global vat tab
|
|       Args:
|         tab_type: a specifier of the target global var tab
|
|       Returns:
|         the index of the new field (None if failed)
|
|   add_regrun_record(self) -> 'int'
|       add new record in regression run configs
|
|       Returns:
|         return the index of added record
|
|   add_tree_node(self, module_path: str, type_name: str, node_name: typing.Union[str, NoneType]=None, pos: typing.Union[list, NoneType]=None) -> 'int'
|       add a tree node
|
|       Args:
|         module_path: path of module associated with the tree node
|         type_name: library type associated with the tree node
|         node_name: user-defined name (default: None, and the name is decided via preference)
|         pos: a list of two floating point numbers containing X, and Y positions
|
|       Returns:
|         A unique ID (UID) of type int representing created node

```

Available Commands

Available commands of DDT under batch mode are as follows:

add_customized_lib(self, filename)

Add customized library defined by users written in Python.

Parameters: `filename` (`str`) – file name of the customized library.

add_global_var_field(self, tab_type)

Add an empty field in global vat tab.

Parameters: `tab_type` (`GlobalVarTabType`) – a specifier of the target global var tab.

Return type: `int`

Returns: The index of the new field (none, if failed)

add_tree_node(self, module_path, type_name, node_name=None, pos=None)

Add a tree node

Parameters:

- **module_path** (`str`) – path of module associated with the tree node. Giving empty string looks up for `module_path` via the given `type_name`.
- **type_name** (`str`) – library type associated with the tree node.
- **node_name** (`Optional[str]`) – user-defined name (default: `None`, and the name is decided via preference)
- **pos** (`Optional[List[float]]`) – a list of two floating point numbers containing X, and Y positions.

auto_layout(self)

Auto layout to rearrange the nodes on the canvas.

batch_run(self, summary_file=None, report_file=None, target=None)

Run DDT session based on data defined in the session.

Parameters:

- **summary_file** (`Optional[str]`) – file name of the generated summary log file.
- **report_file** (`Optional[str]`) – file name of the generated report log file.
- **target** (`Optional[str]`) – path for output files.

Return type: `bool`

Returns: Return `True` if batch run finish successfully.

close(self)

Close and exit DDT.

collapse_all_nodes(self)

Collapse entire forest.

collapse_node(self, uid)

Collapse a subtree rooted at a node.

Parameters: `uid` (`int`) – UID of node to be collapsed.

Return type: `bool`

Returns: Return `True` if collapse nodes successfully.

connect_tree_node(self, src_uid, tgt_uid)

Connect input port of the source node to output port of the target node.

Parameters:

- `src_uid` (`int`) – UID of source node, that is the child node in the tree.
- `tgt_uid` (`int`) – UID of target node, that is the parent node in the tree.

Return type: `bool`

Returns: Return `True` if connect nodes successfully.

decode_session(self, output_file, source_file=None, detail=False)

Decode current session file and dump the content in plain text to a file.

Parameters:

- `output_file` (`str`) – Output file name of decoded info.
- `source_file` (`Optional[str]`) – Decode the given session file instead of current session.
- `detail` (`bool`) – Set `True` to decode every detail info.

Return type: `bool`

Returns: Return `True` if connect nodes successfully.

delete_ddt_nodes(self, uid_list)

Delete a list of tree nodes.

Parameters: `uid_list` (`List[int]`) – A list node UIDs to be deleted.

delete_global_var_field(self, tab_type, field_idx)

Remove field(s) from global var tab.

Parameters:

- `tab_type` (`GlobalVarTabType`) – A specified of the target global var tab.
- `field_idx` (`List[int]`) – A list of field indices to be deleted.

Return type: `bool`

Returns: Return `True` if deletes field(s) successfully.

delete_regrun_record(self, idx_list)

Delete a record from regression run configs.

Parameters:

- `idx_list` (`List[int]`) – List of record indices to be deleted.

disconnect_tree_node(self, src_uid, tgt_uid)

Disconnect input port of the source node to output port of the target node.

Parameters:

- `src_uid` (`int`) – UID of source node, that is the child node in the tree.
- `tgt_uid` (`int`) – UID of target node, that is the parent node in the tree.

Return type: `bool`

Returns: Return `True` if disconnect nodes successfully.

duplicate_ddt_nodes(self, uid_list)

Duplicate a list of nodes. If two nodes have connections, corresponding duplicated nodes have connections as well.

Parameters:

- `uid_list` (`List[int]`) – A list node UIDs to be duplicated.

exec_command(self, cmd)

Execute the given command string.

Parameters:

- `cmd` (`str`) – One-line command string.

Return type: `bool`

Returns: Return value after executing the given command string.

expand_all_nodes(self)

Expand entire forest.

expand_node(self, uid)

Expand a collapsed node.

Parameters:

- `uid (int)` – UID of node to be expanded.

Return type: bool

Returns: Return value after executing the given command string.

export_regrun_record(self, filename)

Export regression run records.

Parameters:

- `filename (str)` – Output filename of exported records.

gen_lib_template_to_file(self, filename)

Generate customized library template to a file.

Parameters:

- `filename (str)` – File name of the generated customized library template.

get_all_nodes(self)

Return all nodes' UID.

Return type: List[int]

get_global_var_count_in_tab(self, tab_type)

Get the number of fields in the global variable tab.

Parameters:

- `tab_type (GlobalVarTabType)` – A specified type of the target global var tab.

Return type: int

get_global_var_tab_field(self, tab_type, index)

Get global var value in a tab.

Parameters:

- `tab_type (GlobalVarTabType)` – A specified type of the target global var tab.
- `index (int)` – Field index to get.

Return type: int

get_node_child(self, uid)

Return a list of node UIDs that are the child of the given node.

Parameters:

- `uid (int)` – Node UID.

Return type: `List[int]`

get_node_config_by_arg(self, uid, arg_name)

Get node config of a specific argument.

Parameters:

- `uid (int)` – Node UID.
- `arg_name (str)` – Argument name of the node config (one of the parameter in method `execute()` of associated library).

Return type: `Optional[str]`

get_node_description(self, uid)

Get node's description.

Parameters:

- `uid (int)` – Node UID.

Return type: `str`

get_node_name(self, uid)

Get node name.

Parameters:

- `uid (int)` – Node UID.

Return type: `str`

get_node_parent(self, uid)

Return a list of nodes UID that is the parent of the given node.

Parameters:

- `uid (int)` – Node UID.

Return type: `List[int]`

get_node_pos(self, uid)

Return node X-Y position in a list of floating points [x, y].

Parameters:

- `uid (int)` – Node UID.

Return type:`List[int]`

get_node_property(self, uid, name)

Get node property.

Parameters:

- `uid (int)` – Node UID.
- `name (str)` – name of the property to get.

get_node_size(self, uid)

Return node size in a list of floating points [width, height] indicating the width and height of this node.

Parameters:

- `uid (int)` – Node UID.

Return type:`List[int]`

get_regrun_record(self, row_idx=None)

Return regression run config, where each row is a dict of {"global var name": "value"}.

Parameters:

- `row_idx (Optional[int], default: None)` – If specified (not None), give the record at that row rather than entire table.

Return type:`Union[List, Mapping]`

import_regrun_record(self, filename)

Import regression run records.

Parameters:

- `filename (str)` – Input filename of imported records.

Return type:`bool`

Returns: Return `True` if import records successfully.

load_preference(self, conf_file)

Load preference file.

Parameters:

- `conf_file (str)` – Preference file name.

move_ddt_node(self, uid, pos)

Move a node to given position.

Parameters:

- `uid (int)` – UID of node to be moved.
- `pos (List[float])` – A list `[x_pos, y_pos]` of the new position, where `x_pos` and `y_pos` are float.

Return type:bool

Returns: Return `True` if node is moved to desired position.

redo(self)

Redo the latest operation.

Return type:bool

Returns: Return `True` if import redo operation successfully.

register_macro_api(self, name, module_path, func_name)

Register a new macro comm and the registration step locates the function via the given `module_path` and function `name`.

Parameters:

- `name (str)` – Name of the new macro command.
- `module_path (str)` – Module path where the function locates.
- `func_name (str)` – Name of the function.

Return type:bool

Returns: Return `True` for success of this registration.

remove_macro_api(self, name)

Remove a registered macro command.

Parameters:

- **name** (**str**) – Name of this macro command to be removed.

Return type:bool

Returns: Return `True` if remove API successfully.

regression_run(self, config_idx=None, num_jobs=1, summary_file=None, report_file=None, target=None, log_file=None, record_file=None, force=False, throw_exception=False, **kwargs)

Run predefined regression-run configurations.

Parameters:

- **config_idx** (**default: None**) – Index of the desired configuration(s) to run.
None: run all configurations type is int: run specified configuration type is list: run configurations associated with the indices.
- **num_jobs** (**int, default: 1**) – Max number of jobs to run configurations (1 for running sequentially).
- **summary_file** (**Optional[str], default: None**) – File name of the generated summary log file.
- **report_file** (**Optional[str], default: None**) – File name of the generated report log file.
- **target** (**Optional[str], default: None**) – Path for output files.
- **log_file** (**Optional[str], default: None**) – Log file name of regression run.
- **record_file** (**Optional[str], default: None**) – Use the records defined in this file rather than records in the session.
- **force** (**bool, default: False**) – Force to write files even if file exists.
- **throw_exception** (**bool, default: False**) – `True` for throwing exception rather than caught internally.

Return type:bool

Returns: Return `True` if regression run is finished successfully.

replay(self, cmd_file)

Replay command file.

Parameters:

- `cmd_file (str)` – Command file to be played.

restore_global_var_tab(self, tab_type, filename)

Open and load global var tab from a file.

Parameters:

- `tab_type (GlobalVarTabType)` – A specified of the target global var tab.
- `filename (str)` – File name to be restored.

Return type: bool

Returns: Return `True` if restore global variable tab from file successfully.

restore_session(self, filename)

Open and load a session from file.

Parameters:

- `filename (str)` – File name to be restored.

Return type: bool

Returns: Return `True` of restore session successfully.

run_cli(self)

Run in command line interactive mode.

run_gui(self)

Run in GUI mode.

save_global_var_tab(self, tab_type, filename)

Save global var tab to a file in JSON format.

Parameters:

- `tab_type (GlobalVarTabType)` – A specified of the target global var tab.
- `filename (str)` – Output file name of saved global var tab.

Return type: bool

Returns: Return `True` if save global variable tab to a file successfully.

save_session(self, filename)

Save current session to target file.

Parameters:

- `filename` (`str`) – File name of the saved session file.

Return type: `bool`

Returns: Return `True` if save session successfully.

set_configuration(self, settings)

Set additional configurations for DDT.

Parameters:

- `settings` (`Mapping[str, object]`) – A dict consists of configuration name and associated value.

Return type: `bool`

Returns: Return `True` if all the given configurations are set successfully.

set_global_var_tab_field(self, tab_type, index, var_name, value, add_quote=False)

Set global var value in a tab.

Parameters:

- `tab_type` (`GlobalVarTabType`) – A specified of the target global var tab.
- `index` (`int`) – Field index to be set.
- `var_name` (`str`) – Name of the global variable.
- `value` (`object`) – Value of the global variable.
- `add_quote` (`bool`) – `True`, if value has extra outermost quote (useful when type of value is string).

Return type: `bool`

Returns: Return `True` if set global variable successfully.

set_node_color(self, uid, red, green, blue)

Set node's color via RGB.

Parameters:

- `uid (int)` – Node UID.
- `red (int)` – Red part of the new color, the value should be in range [0, 255].
- `green (int)` – Green part of the new color, the value should be in range [0, 255].
- `blue (int)` – Blue part of the new color, the value should be in range [0, 255].

Return type: bool

Returns: Return `True` if set node's color successfully.

`set_node_config_by_arg(self, uid, arg_name, value, add_quote=False)`

Set node config of a specific argument. This operation can be undo/redo.

Parameters:

- `uid (int)` – Node UID.
- `arg_name (str)` – Argument name of the node config (one of the parameter in method `execute()` of associated library).
- `value` – Value of the argument (become the argument passed to method `execute()`).
- `add_quote (bool)` – `True` if value has an extra outermost quote (useful when type of value is string).

Return type: bool

Returns: Return `True` if set value to arg in node's config successfully.

`set_node_description(self, uid, value)`

Set node description. This operation cannot be undo.

Parameters:

- `uid (int)` – Node UID.
- `value (str)` – Description of the node.

Return type: bool

Returns: Return `True` if update node description successfully.

`set_node_name(self, uid, value)`

Set node name. This operation cannot be undo.

Parameters:

- `uid` (`int`) – Node UID.
- `value` (`str`) – Name of the node.

Return type: `bool`

Returns: Return `True` if update node name successfully.

`set_node_property(self, uid, name, value, push_undo=True)`

Set node property.

Parameters:

- `uid` (`int`) – Node UID.
- `name` (`str`) – Name of the property to be set.
- `value` – Value of the specified property.
- `push_undo` (`bool`) – `True/False` to enable/disable undo and redo of the API call.

Return type: `bool`

Returns: Return `True` if update node property successfully.

`set_node_size(self, uid, width=0.0, height=0.0)`

Set node size.

Parameters:

- `uid` (`int`) – Node UID.
- `width` (`float`) – New node width (larger than 0), 0 for keeping it unchanged.
- `height` (`float`) – New node height (larger than 0), 0 for keeping it unchanged.

Return type: `bool`

Returns: Return `True` if update node size successfully.

`undo(self)`

Undo previous operation.

Return type: `bool`

Returns: Return `True` if undo previous operation successfully.

update_regrun_record_by_idx(self, row_idx, tab_type, idx, data, add_quote=False)

Update the value of regression run configs at the given cell, which is specified via 1. row: by `row_idx` 2. column: by `tab_type` & `index` => redefine the value of a specific global variable.

Parameters:

- `row_idx` (int) – Index of record to be updated.
- `tab_type` (GlobalVarTabType) – Global variable tab.
- `idx` (int) – Index of the global variable in the tab.
- `data` – Value to update or `None` to remove content at the specified cell.
- `add_quote` (bool, default: False) – True if data has extra outermost quote (useful when type of value is string).

update_regrun_record_by_name(self, row_idx, var_name, data, add_quote=False)

Update the value of regression run configs at the given cell, which is specified via `var_name` => redefine the value of a specific global variable by variable name.

Parameters:

- `row_idx` (int) – Index of record to be updated.
- `var_name` (str) – Variable name to access column.
- `add_quote` (bool, default: False) – True if data should have extra outermost quote (useful when type of value is string).

update_regrun_setting(self, setting)

Update regression run settings.

Parameters:

- `setting` (Mapping[str, object]) – A dict of setting key and associated values.

Command File

In Verdi, every operation is recorded to a **Tcl** command file and each operation in DDT GUI mode is recorded to a DDT command file. The recorded command file name is `ddt_history.log` under the current directory.

The recorded DDT command file is played by DDT, that helps us to reproduce the steps in GUI and batch mode:

```
%> ddt gui --cmdfile ddt_history.log &
```

or

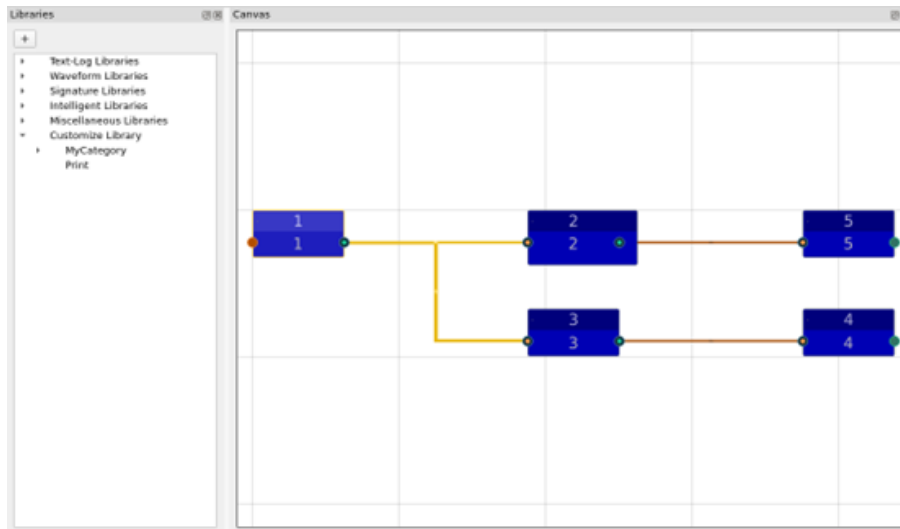
```
%> ddt batch --cmdfile ddt_history.log &
```

This capability allows you to generate a DDT session through scripts. Furthermore, all the valid Python syntax and libraries are available in the DDT command file.

For example,

```
import os
def add_lib():
    root = os.path.join("userlib")
    root = os.path.abspath(root)
    for name in ["util", "use_util"]:
        lib = os.path.join(root, name)
        add_customized_lib(lib)
def create_node(idx, pos):
    node = add_tree_node("verdilearn.ddt.treelib.textlog",
"FindAttributeEvent", pos=pos)
    set_node_name(node, str(idx))
    return node
def main():
    add_lib()
    n1 = create_node(1, [0, 0])
    n2 = create_node(2, [1, 1])
    n3 = create_node(3, [2, 2])
    n4 = create_node(4, [3, 3])
    n5 = create_node(5, [4, 4])
    #-----
    # new line represents the end of this function
    # use comment to separate instead
    #-----
    connect_tree_node(n2, n1)
    connect_tree_node(n3, n1)
    connect_tree_node(n4, n2)
    delete_ddt_nodes([n2])
    undo()
    # undo 13: delete nodes
    connect_tree_node(n5, n2)
    disconnect_tree_node(n4, n2)
    connect_tree_node(n4, n3)
    auto_layout()
    # set node size on tree node
    set_node_size(n2, 300, 150)
    save_session("replay.json")
main()
```

It generates a DDT session as shown in the following image:



Demo Examples

Run demo examples by `ddt demo <case>`. Refer to `ddt demo --help` for the list of cases.

Customized Library

Customized Library contains the following sections:

Copy Folder

Copy demo case folder to current directory. Set environment variable `DDT_WORK_ROOT` to the path of customized library. (optional)

```
cp -rL $VERDI_HOME/share/NPI/python/verdilearn/ddt/demo/customized_lib .
cd customized_lib
export DDT_WORK_ROOT=`pwd`
```

Run Session

Run DDT session in command-line and show summary log. Refer to `ddt summary --help` for other formats.

```
ddt run dvfs_c_ddt.json
ddt summary summary.log --format report | tee dvfs_c_ddt.rpt
ddt summary summary.log --format log | tee -a dvfs_c_ddt.rpt
```

View Tree

Open DDT GUI and load session.

```
ddt gui dvfs_c_ddt.json &
```

Text-Log Parse

Text-Log contains the following sections:

Copy Folder

Copy demo case folder to current directory.

```
cp -rL $VERDI_HOME/share/NPI/python/verdilearn/ddt/demo/textlog_parse .  
cd textlog_parse
```

Run Session

Run DDT session in command-line and show summary log.

```
ddt run dvfs_ddt.json  
ddt summary summary.log --format report | tee dvfs_ddt.rpt  
ddt summary summary.log --format log | tee -a dvfs_ddt.rpt
```

View Tree

Open DDT GUI and load session.

```
ddt gui dvfs_ddt.json &
```

Waveform Extraction

Waveform Extraction contains the following sections:

Copy Folder

Copy demo case folder to current directory.

```
cp -rL  
$VERDI_HOME/share/NPI/python/verdilearn/ddt/demo/waveform_extract .  
cd waveform_extract
```

Run Session

Run DDT session in command-line and show summary log. Refer to `ddt summary --help` for other formats.

```
ddt run fsdb_debug_ddt.json  
ddt summary summary.log --format report | tee fsdb_debug.rpt  
ddt summary summary.log --format log | tee -a fsdb_debug.rpt
```

View Tree

Open DDT GUI and load session.

```
ddt gui fsdb_debug_ddt.json &
```